Transactra

Every transfer balanced. Every action traced.

Software Requirements Specification (SRS)

A Database-First Banking Simulation

CSAI_202_Course_Project

Introduction to Database Systems

Zewail City of Science and Technology

School of Computational Sciences and Artificial Intelligence (CSAI)

Version 1.0

Team Members:

	Name	Email	Section	ID
1	Mohammed Ayman Elboghdadi	s-mohammed.elboghdadi@zewailcity.edu.eg	2	202300888
2	Youssef Mohamed Mahmoud	s-youssef.mawla@zewailcity.edu.eg	5	202505316
3	Ammar Khaled Mostafa	s-ammar.mostafa@zewailcity.edu.eg	5	202500955

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for **Transactra**, a database-first web application that simulates the internal account and payment system of a small bank. It is the single reference used by the team to design the database and the web application, to implement them, and to test them for the CSAI 202 course project.

Specifically, this SRS:

- Specifies 30 functional requirements (FR-01–FR-30) in six functional areas: access and user management, customer onboarding (KYC), account management, funds transfer and ledger, fraud monitoring and compliance, and audit, reporting and integrity.
- Defines 11 non-functional requirements (NFR-01–NFR-11) with measurable targets, verification methods, and acceptance criteria.
- Documents four use-case diagrams and 26 use-case descriptions (UC-01–UC-26) written with the IEEE 830 standards template.
- Provides the conceptual class model (22 classes) and a data dictionary that map directly to SQL Server tables, keys, and constraints.

Intended audience: the project team (database and application developers, testers), the course instructor and teaching assistants who evaluate the project, and anyone who later extends the system.

1.2 Scope

Transactra lets customers apply for an account, hold checking, savings, or wallet accounts, and send simulated money (EGP) to other Transactra accounts. Operations officers review applications and run a mock watchlist screening; compliance analysts investigate alerts produced by explainable fraud rules; administrators manage users, limits, and rules, and review reports, reconciliation results, and audit history.

The main objective is not to imitate every banking service but to show that critical financial rules are enforced by the database itself:

- A transfer is recorded completely or not at all.
- Every completed transfer posts one debit and one matching credit of the same amount.
- Retrying the same request never creates a second transfer.
- Concurrent requests never spend more money than an account is allowed to spend.
- Every important decision and financial action can be traced to an actor, a time, and an outcome.

1.2.1 In Scope

- Access and user management: registration, login/logout, role-based access, staff user administration (FR-01–FR-04).
- Customer onboarding: synthetic application and document metadata, status tracking and correction, review queue, mock watchlist screening, and decisions (FR-05–FR-09).
- Account management: opening checking/savings/wallet accounts, status changes, balance viewing, risk tiers and transfer limits (FR-10–FR-13).
- Funds transfer and ledger: synthetic deposits, internal transfers, validation, atomic execution, double-entry posting, idempotency, controlled reversal, and history/receipts (FR-14–FR-21).
- Fraud monitoring and compliance: configurable rules, alerts, alert review, and case management (FR-22–FR-25).
- Audit, reporting and integrity: audit history, management reports, reconciliation, historical queries, and synthetic data generation (FR-26–FR-30).

1.2.2 Out of Scope

- Real money, real bank accounts, payment networks, and transfers to other banks.
- Real identity verification, real document images, and real sanctions or watchlists.
- Legal or regulatory compliance decisions and production-grade fraud detection, including machine-learning models.
- Multiple currencies, fees, interest accrual, loans, and cards.
- Business customers (KYB) and beneficial-ownership chains (candidate future extension).
- Native mobile apps, email/SMS notifications, and public APIs.

Transactra is standalone. It follows the MVC pattern on the application side and keeps financial business logic in Microsoft SQL Server stored procedures, views, and constraints.

1.2.3 Target Usage Scenarios

The following end-to-end story is the backbone of the final demonstration:

- **Onboarding:** Mona registers and submits an application with synthetic identity details and document metadata. Karim, an operations officer, runs watchlist screening, finds no match, approves the application as Low risk / Tier 2, and opens a checking account.
- **Funding and transfer:** Karim records a synthetic salary deposit of EGP 12,000. Mona sends EGP 500 to her friend Omar; one transfer and two ledger entries (debit Mona, credit Omar) are committed together.
- **Retry and concurrency:** Mona's browser re-sends the same request with the same idempotency key; Transactra returns the original result instead of creating a second transfer. Two simultaneous transfers from her account never overdraw it.

- **Suspicious pattern:** five synthetic accounts send money to the same beneficiary within an hour. A fan-in rule raises an alert; Laila, a compliance analyst, opens a case, records notes, and closes it with a resolution.
- **Oversight:** a manager runs reconciliation (no exceptions), checks Mona's balance at a past date, and reviews the daily dashboard.

These scenarios motivate the requirements in Section 3.1 and are the primary flows of UC-05, UC-09, UC-14, UC-15, UC-19, UC-21, UC-23, UC-24, and UC-25.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition (Transactra-specific)
Account	A container of simulated EGP funds owned by one customer, of exactly one type: Checking, Savings, or Wallet. The internal Bank Funding account has no customer owner.
Alert	A record created when a fraud rule is satisfied. It identifies the rule, the account, and the transfers involved; it is not proof of fraud.
Available balance	The amount a customer may spend now: ledger balance plus the overdraft limit (checking only).
Compliance case	An investigation that groups one or more alerts, with assignment, notes, status history, and a resolution.
Deposit	A synthetic incoming payment (e.g., salary) recorded by an operations officer as a transfer from the Bank Funding account.
Double-entry	Bookkeeping rule: each completed transfer posts one debit and one credit of the same amount and currency.
Idempotency key	A unique value sent with a transfer request so that a repeated request returns the original result instead of executing again.
KYC	Know Your Customer: verifying who a customer is before opening accounts. In Transactra it is simulated with synthetic data.
KYC tier	A level (Tier 1–3) assigned at approval that sets the customer's maximum single-transfer amount and daily transfer limit.
Ledger entry	An immutable debit or credit posted to one account as part of one transfer.
Reconciliation	A check that stored balances equal ledger-derived balances and that every transfer's entries balance.
Reversal	A new, compensating transfer that cancels the financial effect of a completed transfer; the original records are never edited.
Screening	A mock comparison of an applicant's normalized name against a fictional watchlist.
Transfer	A request to move an amount between two Transactra accounts. Types: Internal, Deposit, Reversal. Status: Completed, Rejected, or Reversed.
AML	Anti-Money Laundering.

Term / Acronym	Definition (Transactra-specific)
ACID	Atomicity, Consistency, Isolation, Durability: the properties of a database transaction.
FR / NFR / UC	Functional Requirement / Non-Functional Requirement / Use Case (identifiers in Section 3).
MVC	Model–View–Controller: the architectural pattern of the web application.
RBAC	Role-Based Access Control.
SSMS	SQL Server Management Studio.
T-SQL	Transact-SQL, the SQL dialect of Microsoft SQL Server.
usp_ / vw_	Prefixes of stored procedures and views in the Transactra database.

1.4 References

[1] IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, 1998.

[2] CSAI 202 – Introduction to Database Systems, “Course Project Description and Guidelines,” Zewail City of Science and Technology, School of Computational Sciences and Artificial Intelligence, Fall 2026.

[3] Transactra Project Proposal: A Database-First Banking Simulation for Financial Integrity and Fraud Monitoring, project team, October 2026.

[4] IEEE-SRS Markdown Template, GitHub. [Online]. Available: https://github.com/cemantix-org/ieee-srs/blob/master/IEEE-SRS.md

[5] PlantUML, “Use Case and Class Diagram Documentation.” [Online]. Available: https://plantuml.com (used for Figures 1–7).

[6] Microsoft Learn, “Transaction locking and row versioning guide.” [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide

[7] Microsoft Learn, “Temporal tables.” [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/tables/temporal-tables

[8] Microsoft Learn, “Ledger overview.” [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/security/ledger/ledger-overview

[9] Microsoft Learn, “Introduction to Identity on ASP.NET Core.” [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity

1.5 Overview

Section 1 introduces the purpose, scope, terms, and references. Section 2 describes Transactra’s context, functions, user classes and permissions, stakeholders, environment, constraints, documentation, assumptions and dependencies, and apportioning. Section 3

specifies the requirements: functional requirements (3.1), use cases (3.2), the conceptual class model (3.3), non-functional requirements (3.4), and external interfaces (3.5). Section 4 contains the data dictionary (Appendix A) and glossary (Appendix B).

1.6 Document Conventions

- “Shall” marks a mandatory requirement; “should” a recommended one that may be waived with a documented reason; “may” an optional capability.
- Identifiers FR-xx, NFR-xx, UC-xx, CR-x (common rules), C-xx (constraints), A-xx (assumptions), and D-xx (dependencies) are stable and used for traceability.
- Priorities: High = Must-have, Medium = Should-have, Low = Optional, matching the proposal’s M/S/O scale [3].
- All amounts are in Egyptian pounds (EGP) with two decimal places. All times are server time (Africa/Cairo).
- Status values are written in display form (e.g., Under Review); Appendix A lists the stored codes (e.g., UnderReview). Result codes such as INSUFFICIENT_FUNDS are written exactly as returned by the database.

2. Overall Description

This section gives the background needed to understand the requirements in Section 3. In line with IEEE Std 830-1998 it does not add requirements; where it implies a rule, the formal requirement is referenced.

2.1 Product Perspective

Transactra is a new, self-contained system; it does not replace or extend an existing product, and no legacy data is migrated. Existing open-source projects (e.g., LedgerShield, Banking System DotNet, OpenSourceBank) demonstrate similar ideas; Transactra is designed and implemented independently, and its name was changed from “LedgerGuard” because other projects already use that name.

The system has four logical layers (Figure 1):

- **Presentation layer:** Razor views in the browser for customers and staff.
- **Application layer:** ASP.NET Core MVC controllers and services for input validation, authentication, sessions, role checks, and workflow coordination.
- **Database business-logic layer:** stored procedures, views, and constraints that enforce financial rules and transactions.
- **Data layer:** SQL Server tables for customers, applications, accounts, transfers, ledger entries, alerts, cases, and audit events.

The database is the authoritative record. The website calls approved stored procedures for every financial write and connects with a least-privilege login that cannot modify ledger or audit tables directly (C-03, C-04). Transactra has no external system interfaces: no payment network, bank, identity-verification, or e-mail service is part of the boundary.

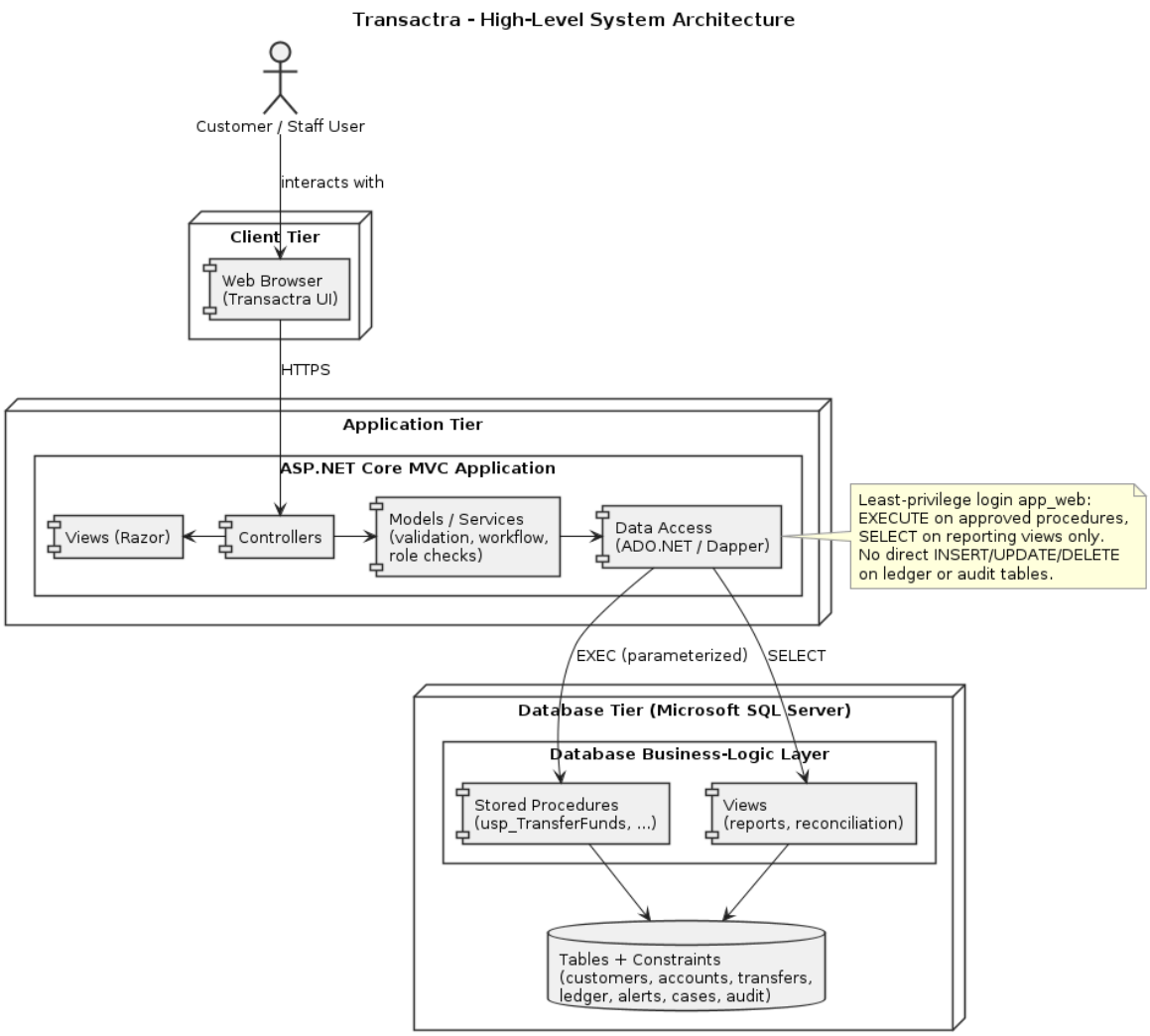


Figure 1: Transactra High-Level System Architecture

2.2 Product Functions

Transactra provides six functional areas. Each is refined into numbered requirements in Section 3.1.

	Functional Area	Summary of Major Functions	FRs	Priority
1	**Access & User Management**	Customer registration, login/logout with lockout, role-based access to screens and actions, staff user and role administration.	FR-01 – FR-04	High
2	**Customer Onboarding (KYC)**	Synthetic application with document metadata, status tracking and correction, review queue, mock watchlist screening, approve/reject/return decisions.	FR-05 – FR-09	High

	Functional Area	Summary of Major Functions	FRs	Priority
3	**Account Management**	Open checking/savings/wallet accounts, freeze/unfreeze/close, view balances, configure risk tiers and transfer limits.	FR-10 – FR-13	High
4	**Funds Transfer & Ledger**	Synthetic deposits, internal transfers with validation, atomic double-entry posting, idempotency, controlled reversal, history and receipts.	FR-14 – FR-21	High
5	**Fraud Monitoring & Compliance**	Configurable explainable rules, automatic alerts, alert review, compliance case workflow.	FR-22 – FR-25	Medium
6	**Audit, Reporting & Integrity**	Append-only audit history, management reports, reconciliation and integrity checks, point-in-time queries, synthetic data generation.	FR-26 – FR-30	High / Medium / Low

Deferred to future versions: transfers to other banks, multiple currencies, fees and interest, business customers (KYB), scheduled/recurring transfers, e-mail/SMS notifications, statement PDF export, mobile apps, and machine-learning fraud scoring.

2.3 User Classes and Characteristics

Transactra has five user classes. Staff roles may be combined in one user (e.g., Compliance Analyst and Administrator in a small team), but a customer user cannot hold a staff role, which keeps customers and the staff who review them separate (FR-03).

User Class	Description	Expertise	Frequency	Key Privileges
Guest	Unauthenticated visitor.	Basic web use.	Once (registration).	Register; log in.
Customer	Registered person who applies for and uses accounts.	Basic web use; no banking or database knowledge.	Daily to weekly.	Own application, accounts, transfers, history, receipts.
Operations Officer	Bank staff who onboard customers and service accounts.	Familiar with KYC steps.	Daily, task-driven.	Review queue, screening, decisions, open/fund accounts, status changes, operational reports.
Compliance Analyst	Bank staff who investigate suspicious activity.	Familiar with AML concepts.	Daily.	Alerts, cases, case-related accounts/transfers/ audit, freeze accounts under investigation.

User Class	Description	Expertise	Frequency	Key Privileges
Administrator (Manager)	Manager responsible for oversight and configuration.	Comfortable with reports and settings.	Weekly or on demand.	Users and roles, tiers/limits, fraud rules, reversals, reports, reconciliation, history, audit, seed data.

Training: customers need no training. Staff users receive a short walkthrough using the User Guide (Section 2.7). No user class needs SQL or database knowledge; all tasks are organized by goal (e.g., “Send money”, “Review alert”).

2.3.1 Permission Matrix

The matrix below is enforced on the server and, where practical, by database permissions; hiding a button is never the only control (FR-03, NFR-06).

Action	Customer	Operations	Compliance	Administrator
Submit / track own application	Yes	No	No	No
View accounts and transfers	Own only	Search (read)	Case-related	Yes (reporting)
Review queue, screening, decisions	No	Yes	Read	Read
Open account, record deposit	No	Yes	No	No
Change account status	No	Freeze, unfreeze, close	Freeze only (case-linked)	Yes
Send internal transfer	Own eligible accounts	No	No	No
Reverse a completed transfer	No	No	No	Yes
Review alerts and manage cases	No	No	Yes	Read; decide escalations
Configure tiers, limits, fraud rules	No	No	No	Yes
Manage users and roles	No	No	No	Yes
Reports	No	Operational	Alerts and cases	All
Audit history	No	No	Case-related	All
Reconciliation, historical queries, seed data	No	No	No	Yes
Edit posted ledger entries	No	No	No	No (reversal only)

2.4 Software Stakeholders

The tables distinguish **external** stakeholders (customers and outside parties) from **internal** stakeholders (the bank and the team building Transactra). Scope shows whether a stakeholder drives CSAI 202 requirements or would matter only if Transactra became a production system.

2.4.1 External Stakeholders

	Stakeholder	Point of Interest	Scope
E-01	Prospective customer (Guest)	Simple registration and a clear application status.	CSAI 202
E-02	Customer	Correct balances; transfers that never post partially or twice; privacy of own data.	CSAI 202
E-03	Transfer recipient	Receives exactly the amount sent, visible in history with a reference.	CSAI 202
E-04	Course instructor and TAs	Sound ER model, constraints, procedures, tests, and a working demonstration.	CSAI 202
E-05	Central bank / financial regulator	KYC/AML compliance, regulatory reporting, record retention.	Production
E-06	External auditors	Reconciled books and tamper-evident audit history.	Production
E-07	Payment networks and other banks	Standard messaging, settlement, and dispute handling for external transfers.	Production
E-08	Identity-verification and sanctions-list providers	Integration contracts, data protection, list update frequency.	Production
E-09	Accessibility users	Screen-reader and keyboard support (partially covered by NFR-10).	Production

2.4.2 Internal Stakeholders

	Stakeholder	Point of Interest	Scope
I-01	Bank management (Administrator)	Oversight dashboards, reconciliation, configuration of limits and rules.	CSAI 202
I-02	Operations team	Efficient application queue, explainable screening, account servicing.	CSAI 202
I-03	Compliance team	Explainable alerts and complete, auditable case history.	CSAI 202
I-04	Database developers / DBA	Normalized schema, constraints, procedures, indexes, permissions.	CSAI 202
I-05	Application developers	Clear stored-procedure contracts and a clean MVC structure.	CSAI 202
I-06	QA / test engineers	Repeatable tests for rollback, idempotency, concurrency, permissions.	CSAI 202

	Stakeholder	Point of Interest	Scope
I-07	Project lead / product owner	Scope, priorities, schedule, integration of team work.	CSAI 202
I-08	Information security team	Penetration testing, key management, incident response.	Production
I-09	Customer support	Disputes, reversal requests, account-access problems.	Production
I-10	Legal and data-protection officer	Retention rules and compliance with Egypt's Personal Data Protection Law (No. 151 of 2020).	Production

Note: only stakeholders marked "CSAI 202" generate requirements in this SRS. "Production" stakeholders are recorded to show what a real deployment would add; they do not expand the course scope.

2.5 Operating Environment

2.5.1 Client Environment

- Google Chrome, Mozilla Firefox, and Microsoft Edge (latest stable versions at test time) with JavaScript enabled.
- Viewports ≥ 360 × 640 px (mobile) and ≥ 1366 × 768 px (desktop); no plug-ins required.

2.5.2 Application Server

- ASP.NET Core 8 (LTS) MVC application hosted on Kestrel (or IIS Express during development) on Windows 10/11 or Linux.
- HTTPS with a development certificate for local demonstrations.

2.5.3 Database Server

- Microsoft SQL Server 2022 Developer or Express edition (SQL Server 2019 is acceptable; ledger tables then become unavailable, see C-16).
- SQL Server Management Studio for running scripts, inspecting data, and reading execution plans.
- Two logins: a deployment login (db_owner) used only to run scripts, and the website login app_web with least privilege (C-04).

2.5.4 Network

- Local machine or classroom LAN; the browser reaches the application over HTTPS and the application reaches SQL Server over TCP 1433 (or shared memory on the same machine). The database is not reachable from the browser.

2.5.5 Expected Data Scale

- Demonstration dataset (FR-30): about 200 customers, 300 accounts, 10,000 transfers (20,000 ledger entries) over 30 days, with planted suspicious patterns.
- Benchmark dataset: up to 100,000 transfers for index and concurrency experiments (NFR-09).

2.6 Design and Implementation Constraints

ID	Constraint	Description	Related
C-01	Course Technology Stack	Microsoft SQL Server with T-SQL for the database; C# with ASP.NET Core MVC for the backend; HTML, CSS, JavaScript and Bootstrap for the frontend, as approved by the instructor.	D-08
C-02	MVC + Database Logic Layer	Controllers, Views, and Models are separated in the application; financial rules live in stored procedures, views, and constraints, not in Views or Controllers.	2.1
C-03	Procedure-Only Financial Writes	Transfers, ledger entries, deposits, reversals, idempotency records, account status changes, and audit events are written only through approved stored procedures.	FR-17, NFR-01
C-04	Least-Privilege Database Login	The website login has EXECUTE on approved procedures and SELECT on reporting views only; it is denied INSERT/UPDATE/DELETE on ledger and audit tables.	NFR-05
C-05	Exact Money Type	Monetary values use DECIMAL(18,2); FLOAT, REAL, and MONEY are not used.	NFR-02
C-06	Synthetic Data Only	All people, identity numbers, documents, watchlist entries, and transactions are fictional; no real identity images are collected or stored.	NFR-07
C-07	No Real Money or External Services	No payment network, bank, KYC API, sanctions list, paid API, or real credential is used; the demonstration runs offline.	1.2.2
C-08	Password Storage	Passwords are hashed with ASP.NET Core Identity's salted PBKDF2 hasher; plaintext or reversible storage is prohibited.	FR-01, NFR-05
C-09	Parameterized Database Access	Every database call uses parameters; user input is never concatenated into SQL.	NFR-05
C-10	Validation in Two Places	Input is validated by the application and again by database constraints (CHECK, FOREIGN KEY, UNIQUE) or procedure checks.	NFR-02
C-11	Single Currency and Language	All accounts use EGP; the UI is in English. Names are stored as NVARCHAR so Arabic names are possible.	A-04
C-12	Responsive UI	Bootstrap 5 layout usable on the viewports in 2.5.1.	NFR-10

ID	Constraint	Description	Related
C-13	Safe Error Handling	Procedures return stable result codes; users never see stack traces, SQL errors, or object names.	CR-6
C-14	Explainable Fraud Rules	Fraud detection uses transparent SQL rules with documented parameters; machine-learning models are excluded.	FR-22
C-15	Repository Structure	One GitHub repository with /sql/schema, /sql/procedures, /sql/views, /sql/seed, /tests, /src, /docs, and README.md; SQL scripts are numbered and re-runnable in order.	NFR-11
C-16	Optional SQL Server Features	Temporal and ledger tables are used only where the installed edition supports them; no High-priority requirement depends on them.	FR-29

2.7 User Documentation

- **User Guide** (PDF): one chapter per role — Customer, Operations Officer, Compliance Analyst, Administrator — with screenshots of each task.
- **Demonstration Script:** the step-by-step story in Section 1.2.3, with the expected database results at each step.
- **README.md:** prerequisites and versions, script execution order, how to load seed data, run tests, and start the website, and known limitations.
- **In-application help:** tooltips for terms such as available balance, idempotency (“This request was already processed”), alert, and case status.
- **Database documentation** (team): ER diagram, relational schema, data dictionary (Appendix A), and stored-procedure catalog (Section 3.5.3).

2.8 Assumptions and Dependencies

2.8.1 Assumptions

- **A-01.** Transactra runs as a local or classroom demonstration with synthetic users and money.
- **A-02.** The team (3–4 members) has about 14 weeks and access to SQL Server 2019+ and the .NET 8 SDK.
- **A-03.** Every email identifies exactly one user; staff users are created by an administrator and are never customers.
- **A-04.** All accounts are in EGP; currency conversion is not needed.
- **A-05.** Transfers move money only between Transactra accounts. Deposits come from one internal Bank Funding account, which may hold a negative balance equal to total net deposits.

- **A-06.** Server time (Africa/Cairo) is authoritative; a “day” for limits is 00:00:00–23:59:59 server time.
- **A-07.** A customer has at most one application in progress (Submitted, Under Review, or Returned) at a time.
- **A-08.** A watchlist possible match is not proof of wrongdoing and always requires a human disposition; likewise an alert is a rule firing, not proven fraud.
- **A-09.** Account.balance is a maintained snapshot updated in the same transaction as the ledger; the ledger is the source of truth and reconciliation compares the two.
- **A-10.** Fees, interest accrual, and overdraft interest are not modelled; interestRateLabel is for display only.
- **A-11.** Financial records are never deleted; closed accounts, rejected transfers, and closed cases are retained.
- **A-12.** No regulatory regime binds the course project; a production version would need central-bank and data-protection compliance (E-05, I-10).

2.8.2 Dependencies

- **D-01.** SQL Server 2019 or later (2022 for the optional ledger-table demonstration).
- **D-02.** .NET 8 SDK, ASP.NET Core MVC, and ASP.NET Core Identity.
- **D-03.** Microsoft.Data.SqlClient (ADO.NET) or Dapper for calling stored procedures.
- **D-04.** Bootstrap 5 for layout and Chart.js for dashboard charts.
- **D-05.** SSMS for scripts and execution plans.
- **D-06.** xUnit for application tests and T-SQL test scripts (tSQLt optional) for database tests.
- **D-07.** GitHub access for all members and the assigned TA.
- **D-08.** Instructor approval of the technology stack and scope before implementation.
- **D-09.** PlantUML or diagrams.net for UML and ER diagrams.

2.9 Apportioning of Requirements

All 30 FRs and 11 NFRs are in scope. Priorities decide what may be dropped if time runs short: every **High** requirement is mandatory for the final delivery; **Medium** requirements are built once the transfer core (Weeks 8–9) is stable; the **Low** requirement (FR-29) is attempted only after every mandatory requirement passes its tests. Work is apportioned to the 14-week plan as follows:

Weeks	Stage	Requirements	Priority
1–7	Requirements and design	This SRS, use cases, ER model, relational mapping, normalization.	—
8	Database foundation	Schema, constraints, indexes, roles; data structures for FR-01–FR-13.	High

Weeks	Stage	Requirements	Priority
9	Transfer core	FR-14, FR-16–FR-19 (procedures and tests); reconciliation query for FR-28.	High
10	Initial interface	FR-01–FR-03, FR-05–FR-07, FR-09, FR-10, FR-12, FR-15, FR-21.	High
11	Compliance features	FR-08, FR-22–FR-26.	Medium
12	Reporting and integration	FR-04, FR-11, FR-13, FR-20, FR-27, FR-28 (UI), FR-30.	High / Medium
13	Testing and evaluation	NFR-01–NFR-11 verification; FR-29 if time allows.	Low (FR-29)
14	Final submission	README, final report, demonstration.	—

Items deferred to future versions (Section 2.2) are not traced to any FR or use case and are excluded from acceptance testing.

3. Specific Requirements

This section states Transactra's requirements in enough detail to design, implement, and test the system: functional requirements (3.1), use cases (3.2), the conceptual class model (3.3), non-functional requirements (3.4), and external interfaces (3.5).

3.1 Functional Requirements

Each requirement lists its inputs, processing rules, outputs, priority, and traceability to use cases (3.2), classes (3.3), and the main database object. Rules that apply to every requirement are stated once in 3.1.1 and not repeated.

3.1.1 Common Rules

ID	Rule (applies to every FR unless stated otherwise)
CR-1	**Authentication.** Every function except registration (FR-01) and login requires an authenticated session; other requests are redirected to the login page.
CR-2	**Authorization.** Each action is allowed only to the roles in the permission matrix (2.3.1). The controller checks the role, and the stored procedure re-checks role and ownership using the @ActorUserId parameter (FR-03).
CR-3	**Amounts.** Monetary amounts are DECIMAL(18,2), greater than 0.00 and at most 1,000,000.00 per operation, with at most two decimal places.
CR-4	**Time.** Timestamps are set by the database server (SYSDATETIME(), Africa/Cairo). User-entered dates must be valid calendar dates; dates of birth and issue dates cannot be in the future.
CR-5	**Text.** Input is trimmed. Names are 2–100 characters; a required reason is 10–500 characters; descriptions are at most 140 characters.
CR-6	**Results and errors.** Every procedure returns a result code (e.g., OK, INSUFFICIENT_FUNDS) that the UI maps to a plain message. A failed operation changes nothing and never shows internal details (C-13).
CR-7	**Audit.** Every create, change, decision, financial action, and denied access writes an audit event (FR-26) in the same transaction as the change; denied or failed attempts are written in their own transaction.
CR-8	**Concurrent edits.** Workflow records (applications, accounts, cases, rules, tiers) carry a rowversion. If a record changed after the user loaded it, the update is refused with "This record was changed by another user — reload and try again".

3.1.2 Detailed Functional Requirements

Area 1 — Access and User Management

FR-01 — Customer Registration

Description	The system shall allow a guest to create a customer user.
Inputs	Full name, email, password, password confirmation, and acknowledgement that Transactra is a simulation using fictional data.
Processing & Rules	1. All fields are required. 2. Email must have a valid format and be unique (case-insensitive); it is stored in lowercase. 3. Password: at least 10 characters with an uppercase letter, a lowercase letter, and a digit; it must match the confirmation and must not contain the email's local part. 4. The password is stored only as a salted hash (C-08). 5. The new user receives the Customer role and status Active. No customer profile or account is created yet (FR-05, FR-10).
Outputs	Success message and redirect to login. A taken email shows "This email is already registered" and creates nothing.
Priority	High
Traceability	UC-01 · User, Role · ASP.NET Core Identity

FR-02 — Login and Logout

Description	The system shall authenticate users, protect accounts against password guessing, and let users end their session.
Inputs	Email and password; Log Out action.
Processing & Rules	1. The password is verified against the stored hash; failure shows one generic message, "Invalid email or password". 2. After 5 consecutive failures the user is locked for 15 minutes (lockoutEndAt); attempts during lockout get the same generic message. A successful login resets the counter. 3. Inactive users cannot log in. 4. On success a new session cookie (HttpOnly, Secure, SameSite=Strict) is issued and the user lands on the role's home page: Customer → My Accounts (or Application Status if no account exists), Operations Officer → Application Queue, Compliance Analyst → Alert Queue, Administrator → Dashboard. Staff created with a temporary password must change it first (FR-04). 5. Sessions expire after 15 minutes of inactivity and 8 hours at most. 6. Logout invalidates the session on the server; protected pages are sent with no-cache headers, so the Back button shows no data.

Outputs	Role home page; login page with “You have been logged out”.
Priority	High
Traceability	UC-02, UC-03 · User

FR-03 — Role-Based Access Control

Description	The system shall restrict every screen, action, and record to the roles and ownership rules of the permission matrix (2.3.1).
Inputs	Authenticated user’s roles; requested action and record identifiers.
Processing & Rules	1. Roles: Customer, OperationsOfficer, ComplianceAnalyst, Administrator. A user holds at least one role; the Customer role cannot be combined with a staff role. 2. Menus show only permitted items, but every controller action is protected by a role check, and every procedure verifies role and record ownership (customers: own accounts only; compliance: case-related records only). 3. A request for another customer’s record returns “Not found”, so record existence is not revealed; a request for a forbidden function returns “Access denied”. 4. Role changes take effect on the user’s next request. 5. The database login app_web can only execute approved procedures and read reporting views (C-04).
Outputs	Permitted page, “Access denied”, or “Not found”; audit event for each denial.
Priority	High
Traceability	All UCs (cross-cutting) · User, Role · database role app_web

FR-04 — Manage Staff Users and Roles

Description	The system shall allow an administrator to create staff users, change their roles, and activate or deactivate any user.
Inputs	Name, email, staff roles; activate/deactivate; unlock.
Processing & Rules	1. Only administrators may use this function. 2. A new staff user receives a generated temporary password that must be changed at first login. 3. An administrator cannot remove their own Administrator role or deactivate themselves; at least one active Administrator must always exist. 4. Deactivation ends the user’s sessions immediately. Open cases assigned to a deactivated analyst are marked Unassigned for reassignment. 5. Users are never deleted. Customer users can be deactivated (blocking login) but never given staff roles. 6. An administrator may unlock a locked user.
Outputs	User list with roles and status; confirmation; audit event with roles before and after.
Priority	Medium
Traceability	UC-04 · User, Role, AuditLog · usp_ManageUser

Area 2 — Customer Onboarding (KYC)

FR-05 — Submit Onboarding Application

Description	The system shall allow a customer to submit a synthetic onboarding application with identity-document metadata.
Inputs	Full name, date of birth, mobile number, city, synthetic identity reference number; one or more documents, each with type (National ID card, Passport, Proof of address), reference number, issue date, and expiry date.
Processing & Rules	1. The applicant must be at least 18 years old. 2. Mobile number: 11 digits matching 01[0125]XXXXXXXX. 3. At least one identity document (National ID card or Passport) is required. For every document, issue date ≤ today < expiry date. 4. The customer must have no application in progress and no approved application (A-07). 5. usp_SubmitApplication creates or updates the Customer profile (status Pending) and creates the application (status Submitted, revisionNo 1) with its documents in one transaction. 6. Only metadata is stored; no file upload exists (C-06). The form reminds users to enter fictional data.
Outputs	Application reference (e.g., APP-000123) with status Submitted.
Priority	High
Traceability	UC-05 · Customer, KYCApplication, KYCDocument · usp_SubmitApplication

FR-06 — Track and Correct Application

Description	The system shall show the customer the status and history of their application and allow corrections when it is returned.
Inputs	Corrected application fields and documents (Returned applications only).
Processing & Rules	1. Status lifecycle: Submitted → Under Review → Approved, Rejected, or Returned; a corrected Returned application goes back to Submitted. 2. The customer sees the status, dates, and the officer's reason for a Returned or Rejected decision. Screening results are never shown to customers. 3. Only a Returned application can be edited. Resubmission re-applies all FR-05 rules and increments revisionNo; earlier values remain in the audit history. 4. A Rejected application is final; the customer may submit a new application after 30 days. 5. After approval the page links to the customer's accounts once they are opened.
Outputs	Status page with timeline; resubmission confirmation.
Priority	High
Traceability	UC-06 · KYCApplication, KYCDocument · usp_ResubmitApplication

FR-07 — Review Application Queue

Description	The system shall give operations officers a queue of applications to review and a detailed view of each.
Inputs	Filters (status, date range, assigned to me); Start review; Reassign to me.
Processing & Rules	1. The queue lists Submitted and Under Review applications, oldest first, with reference, applicant name, submission date, revision number, screening state (Not run, Clear, Possible match, Dispositioned), and assigned officer. 2. Start review sets the status to Under Review and assigns the application to the officer. Only the assigned officer may decide; another officer can take over with Reassign to me. 3. The detail view shows applicant data, document metadata with flags (expired, expiring within 30 days), earlier revisions, and screening results. 4. Officers can search customers and accounts by name, email, or account number in read-only mode.
Outputs	Queue, application detail, read-only search results.
Priority	High
Traceability	UC-07 · KYCApplication, Customer, KYCDocument, ScreeningResult · vw_ApplicationQueue

FR-08 — Run Watchlist Screening

Description	The system shall compare an applicant's normalized name with the synthetic watchlist, store the result, and require an officer disposition for every possible match.
Inputs	Application; for each possible match: disposition (FalsePositive or TrueMatch) and reason.
Processing & Rules	1. Normalization: uppercase, remove diacritics and punctuation, collapse repeated spaces. 2. A possible match exists with an active entry when the normalized names are equal, or when at least two name tokens of three or more letters are equal (e.g., "MOHAMED ALI HASSAN" and "ALI HASSAN"). The match reason is stored. 3. The outcome is Clear (one result row) or PossibleMatch (one row per matched entry). 4. Each possible match needs a disposition with a reason before any decision (FR-09). 5. Screening is run again for each new revision; earlier results are kept. 6. Screening data is visible only to staff.
Outputs	Screening outcome, matched entries with reasons, disposition status.
Priority	Medium
Traceability	UC-08 · ScreeningResult, WatchlistEntry · usp_RunScreening, usp_DispositionMatch

FR-09 — Decide on Application

Description	The system shall allow the assigned operations officer to approve, reject, or return an application with a recorded reason.
Inputs	Decision; reason; risk level (Low, Medium, High) and KYC tier (1–3) for approval.
Processing & Rules	1. Allowed only for the assigned officer while the application is Under Review. 2. If screening has not run for the current revision, it runs first (FR-08). 3. Approval requires: every possible match dispositioned, none as TrueMatch, and an unexpired identity document. A TrueMatch allows only Reject. 4. High-risk customers can only be assigned Tier 1. 5. A reason is required for Reject and Return. 6. Effects — Approve: application Approved, customer Active with risk level and tier. Return: application Returned for correction (FR-06). Reject: application and customer Rejected. 7. Decision, decider, and time are immutable once saved.
Outputs	Updated status visible to the customer (FR-06); Open Account becomes available (FR-10).
Priority	High
Traceability	UC-09 · KYCApplication, Customer, KycTier · usp_DecideApplication

Area 3 — Account Management

FR-10 — Open Account

Description	The system shall allow an operations officer to open checking, savings, or wallet accounts for approved customers.
Inputs	Customer; account type; overdraft limit (checking, default 0), interest-rate label (savings), maximum balance (wallet, default 50,000).
Processing & Rules	1. The customer must be Active (approved). 2. Each customer may hold at most one checking, two savings, and one wallet account. 3. The database generates a unique 12-digit account number: a type prefix (10 Checking, 20 Savings, 30 Wallet) followed by a 10-digit sequence. 4. The Account row and its subtype row are created in one transaction with status Active, balance 0.00, currency EGP. 5. An overdraft limit above 0 (maximum 5,000) is allowed only for Tier 2 and Tier 3 customers. 6. Savings accounts use a monthly withdrawal limit of 6 outgoing transfers.
Outputs	Account number shown to the officer and in the customer’s account list (FR-12).
Priority	High
Traceability	UC-10 · Account, CheckingAccount, SavingsAccount, WalletAccount · usp_OpenAccount

FR-11 — Change Account Status

Description	The system shall allow authorized staff to freeze, unfreeze, or close an account with a reason.
Inputs	Account; new status; reason.
Processing & Rules	1. Transitions: Active → Frozen, Frozen → Active, Active or Frozen → Closed (final). 2. Frozen accounts reject customer-initiated outgoing transfers; incoming credits and administrator reversals are still allowed. 3. An account can be closed only with a balance of exactly 0.00 and no open alerts or cases concerning it. 4. Operations officers and administrators may perform any transition. Compliance analysts may only freeze, and only accounts linked to a case assigned to them. 5. The reason, old status, and new status are recorded in the audit history. 6. The status change takes the same account lock as transfers (FR-17), so no transfer completes on an account after its freeze commits. 7. Unfreezing starts the monitoring window of rule R4 (FR-22). 8. The internal Bank Funding account cannot be frozen or closed.
Outputs	Updated status in all views.
Priority	High
Traceability	UC-11 · Account, AuditLog · usp_ChangeAccountStatus

FR-12 — View Accounts and Balances

Description	The system shall show customers their own accounts and balances.
Inputs	None (the authenticated customer).
Processing & Rules	1. For each own account: account number, type, status, balance, available balance, overdraft limit (checking), maximum balance (wallet), and remaining savings withdrawals this month. 2. Available balance = balance + overdraft limit for an Active account, and 0.00 for Frozen or Closed accounts. 3. The page shows the remaining daily transfer allowance of the customer's tier and the last five transfers per account, linking to full history (FR-21). 4. A customer without an approved application sees the application status instead (FR-06). 5. Operations officers see the same view in read-only mode through customer search (FR-07).
Outputs	Account overview.
Priority	High
Traceability	UC-12 · Account, KycTier, Transfer · vw_CustomerAccounts

FR-13 — Configure Risk Tiers and Transfer Limits

Description	The system shall allow an administrator to set each tier's transfer limits and to change an individual customer's tier.
Inputs	Per tier: maximum single transfer and daily transfer limit; customer, new tier, reason.
Processing & Rules	1. Three fixed tiers with default limits: Tier 1 — 5,000 / 10,000; Tier 2 — 50,000 / 100,000; Tier 3 — 250,000 / 500,000 (single / daily). 2. For each tier, max single ≤ daily ≤ 1,000,000; limits never decrease from Tier 1 to Tier 3. 3. Changes apply to transfers submitted after the change commits and never affect completed transfers. 4. Changing a customer's tier requires a reason; High-risk customers remain in Tier 1.
Outputs	Updated tier table; audit event with old and new values.
Priority	Medium
Traceability	UC-13 · KycTier, Customer · usp_UpdateTier, usp_ChangeCustomerTier

Area 4 — Funds Transfer and Ledger

FR-14 — Fund Account (Synthetic Deposit)

Description	The system shall allow an operations officer to record a synthetic incoming payment (e.g., salary) into a customer account.
Inputs	Destination account, amount, description; idempotency key (hidden).
Processing & Rules	1. The destination must be an Active customer account; a wallet's maximum balance applies. 2. The deposit is a transfer of type Deposit from the internal Bank Funding account to the customer account, executed with the same atomic, double-entry, and idempotency rules as FR-17–FR-19. 3. Tier limits do not apply to deposits. 4. Deposits are not evaluated by the fraud rules, which monitor outgoing customer transfers.
Outputs	Reference number and new balance.
Priority	High
Traceability	UC-14 · Transfer, LedgerEntry, Account · usp_RecordDeposit

FR-15 — Submit Internal Transfer

Description	The system shall allow a customer to send money from one of their accounts to another Transactra account.
Inputs	Source account (own), destination account number, amount, optional description; idempotency key generated when the form is displayed.

Processing & Rules	1. Two steps: (a) the customer enters the details and the system shows a confirmation page with the recipient's masked name (e.g., "O*** A***"), account type, amount, and the available balance after the transfer; (b) the customer confirms and the transfer is executed (FR-16, FR-17). 2. The lookup reveals only whether the account exists and the masked name. More than 10 failed lookups in 10 minutes blocks further lookups for 10 minutes. 3. Transfers between the customer's own accounts are allowed. 4. The confirm button is disabled after the first click; a refresh or resubmission reuses the same key (FR-19).
Outputs	Receipt for a Completed transfer, or a plain-language rejection reason.
Priority	High
Traceability	UC-15 · Transfer · usp_LookupAccount, usp_TransferFunds

FR-16 — Validate Transfer Request

Description	The system shall execute a transfer only when every check passes, and otherwise reject it with a reason code.
Inputs	Transfer request from FR-15.
Processing & Rules	Checks, in order, with their rejection codes: 1. Amount valid per CR-3 — INVALID_AMOUNT. 2. Source ≠ destination — SAME_ACCOUNT. 3. Both accounts exist — ACCOUNT_NOT_FOUND. 4. Caller owns the source — NOT_AUTHORIZED. 5. Source is Active; destination is not Closed — ACCOUNT_NOT_ACTIVE. 6. Both accounts in the same currency — CURRENCY_MISMATCH. 7. Amount ≤ tier maximum single transfer — LIMIT_SINGLE_EXCEEDED. 8. Today's completed outgoing transfers + amount ≤ tier daily limit — LIMIT_DAILY_EXCEEDED. 9. Savings source: fewer than 6 outgoing transfers this month — SAVINGS_WITHDRAWAL_LIMIT. 10. Amount ≤ available balance — INSUFFICIENT_FUNDS. 11. Wallet destination: balance + amount ≤ maximum balance — WALLET_MAX_BALANCE. Checks 1–3 also run in the application for quick feedback, but all checks run inside the procedure; checks 5–11 run after the account locks are taken (FR-17), so they see committed values. A rejected request is stored as a Transfer with status Rejected and its code, with no ledger entries.
Outputs	Result code mapped to a user message; Rejected transfer record.
Priority	High
Traceability	UC-15 · Transfer, Account, KycTier · usp_TransferFunds

FR-17 — Execute Transfer Atomically

Description	The system shall execute every transfer, deposit, and reversal in one database transaction inside a stored procedure.
Inputs	Validated request, actor, idempotency key.
Processing & Rules	1. Sequence: begin transaction → claim the idempotency key (FR-19) → lock both account rows with UPDLOCK, ROWLOCK in ascending accountId order → run checks 5–11 (FR-16) → insert the Transfer (Completed) → insert two ledger entries (FR-18) → update both account balances → store the idempotency result → write the audit event → commit. 2. XACT_ABORT is ON and the body is in TRY/CATCH: any error rolls back every step. Business-rule failures then store a Rejected transfer; technical failures store nothing and show “Transfer failed — no money was moved”. 3. Isolation is READ COMMITTED with update locks on the account rows; consistent lock order avoids deadlocks. If a deadlock (error 1205) still happens, the application retries up to 3 times with the same key. 4. Reference numbers have the form TRX-YYYYMMDD-NNNNNN. 5. Fraud evaluation (FR-23) runs after commit; its failure never undoes a transfer. 6. No other code path may insert transfers or ledger entries (C-03).
Outputs	Committed transfer with entries, or no change.
Priority	High
Traceability	UC-14, UC-15, UC-16 · Transfer, LedgerEntry, Account · usp_TransferFunds

FR-18 — Post Double-Entry Ledger Entries

Description	The system shall record every completed transfer as exactly one debit and one matching credit, and keep posted entries unchangeable.
Inputs	Completed transfer inside FR-17.
Processing & Rules	1. A Completed transfer has exactly two entries: a Debit on the source and a Credit on the destination, each equal to the transfer amount, in the same currency, with the same posting time. UNIQUE (transferId, entryType) enforces one of each. 2. Rejected transfers have no entries. 3. Each entry stores balanceAfter, the account’s balance immediately after posting. 4. For every account, balance = Σ credits − Σ debits; across the whole system, Σ debits = Σ credits (checked by FR-28). 5. Entries are immutable: app_web has no UPDATE or DELETE permission, and a trigger rejects any update or delete. Mistakes are corrected only by a reversal (FR-20).
Outputs	Ledger entries visible in history and statements.
Priority	High
Traceability	UC-15 · LedgerEntry, Transfer, Account · usp_TransferFunds, trg_LedgerEntry_Immutable

FR-19 — Prevent Duplicate Requests (Idempotency)

Description	The system shall execute each transfer, deposit, or reversal request at most once, even if it is sent several times.
Inputs	Idempotency key and request details.
Processing & Rules	1. The server generates a key (GUID) when it renders the form; it is unique per user (UNIQUE userId, idempotencyKey). 2. A fingerprint (SHA-256 of operation type, source, destination, amount, description) is stored with the key. 3. Same key and same fingerprint: the stored result (reference, status, code) is returned without executing again — including an earlier rejection — with the message “This request was already processed”. 4. Same key with a different fingerprint: rejected with IDEMPOTENCY_CONFLICT; the original transfer is unchanged. 5. If two requests with the same key arrive together, the unique index and the key lock let only one execute; the other waits and returns the same result. 6. Keys are kept for 24 hours and then purged; replays and conflicts are audited.
Outputs	Original result or conflict message.
Priority	High
Traceability	UC-15 · IdempotencyRecord, Transfer · usp_TransferFunds

FR-20 — Reverse a Completed Transfer

Description	The system shall allow an administrator to cancel the effect of a completed transfer by posting a compensating transfer.
Inputs	Original transfer reference; reason; idempotency key.
Processing & Rules	1. Only Completed transfers of type Internal or Deposit, not yet reversed and at most 30 days old, can be reversed. A reversal cannot itself be reversed. 2. A new transfer of type Reversal moves the same amount from the original destination back to the original source, references the original (reversalOfTransferId), and is posted with FR-17–FR-19. The original’s status becomes Reversed; its ledger entries are untouched. 3. The original destination must have enough available balance and must not be Closed; Frozen accounts may be debited by a reversal. Otherwise the reversal is rejected with REVERSAL_INSUFFICIENT_FUNDS. 4. Tier limits do not apply. Both customers see the reversal in their history.
Outputs	Reversal reference; updated statuses and balances.
Priority	Medium
Traceability	UC-16 · Transfer, LedgerEntry · usp_ReverseTransfer

FR-21 — View Transfer History and Receipts

Description	The system shall show customers the transfers that involve their accounts and provide a receipt for each completed transfer.

Inputs	Filters: account, date range (up to 12 months), amount range, status, direction, reference.
Processing & Rules	1. Lists transfers where one of the customer's accounts is the source or destination, newest first, 25 per page, with direction, signed amount, and the balance after the transfer for the customer's account. 2. Rejected transfers are shown only to the sender, with the reason. 3. Receipt: reference, date and time, source account, destination (masked number ****1234 and masked name), amount, description, status, and a link to any reversal; printable. 4. A counterparty's balance or full details are never shown.
Outputs	History list; receipt.
Priority	High
Traceability	UC-17 · Transfer, LedgerEntry · vw_CustomerTransferHistory

Area 5 — Fraud Monitoring and Compliance

FR-22 — Configure Fraud Rules

Description	The system shall provide explainable fraud rules whose parameters an administrator can change.
Inputs	Rule; active flag; threshold count, threshold amount, window (minutes), severity.
Processing & Rules	1. Four predefined rules (defaults in brackets): • R1 VELOCITY — more than N [5] outgoing transfers from one account within W [10] minutes; Medium. • R2 FAN_IN — at least N [3] different source accounts send to the same destination within W [60] minutes; High. • R3 HIGH_VALUE_WINDOW — outgoing total from one account ≥ A [EGP 40,000] within W [1,440] minutes; Medium. • R4 POST_UNFREEZE — any outgoing transfer within W [1,440] minutes after the account is unfrozen; High. 2. Valid ranges: count 1–100, amount 1–1,000,000, window 1–10,080 minutes. 3. Changes apply to evaluations after the change commits; existing alerts keep the parameters recorded in their summary. 4. New rule types require code changes and are out of scope for configuration.
Outputs	Rule list with parameters and state; audit event with old and new values.
Priority	Medium
Traceability	UC-18 · FraudRule · usp_UpdateFraudRule

FR-23 — Generate Fraud Alerts

Description	The system shall evaluate the active rules and create an alert whenever a rule is satisfied.
Inputs	A completed internal transfer, or a date range for batch evaluation.

Processing & Rules	1. usp_EvaluateFraudRules runs after each completed internal transfer commits, and in batch mode over a date range (for seed data and re-evaluation). 2. An alert records the rule, the account (R1, R3, R4: source; R2: destination), the severity, a summary with the values and thresholds (e.g., “6 transfers in 10 min; threshold 5”), status New, and links to every transfer in the window. 3. Severity is raised one level (up to High) when the customer’s risk level is High. 4. If an alert for the same rule and account is still New or Under Review and its window overlaps, the new transfer is linked to that alert instead of creating a new one. 5. Evaluation runs in its own transaction. A failure is logged and picked up by the next batch run; transfers are never blocked or delayed (detection, not prevention). 6. Customers never see alerts.
Outputs	New or updated alerts in the alert queue (FR-24).
Priority	Medium
Traceability	UC-19 · Alert, FraudRule, Transfer · usp_EvaluateFraudRules

FR-24 — Review Alerts

Description	The system shall let compliance analysts examine alerts and dispose of them or move them into cases.
Inputs	Filters (rule, severity, status, date range, analyst); actions Dismiss and Add to case; reason.
Processing & Rules	1. The queue shows alert ID, rule, severity, account, customer, time, status, and case; by default it shows New alerts, highest severity first, then oldest first. 2. The detail view shows the rule’s description and parameters, the linked transfers (amounts, times, counterparties), the customer’s risk level and tier, and other alerts on the same account in the last 90 days. 3. Dismiss as false positive (reason required): the alert becomes Closed with disposition FalsePositive. 4. Add to case (new or existing open case): the alert becomes Under Review (FR-25). An alert belongs to at most one case. 5. Closed alerts are read-only; administrators have read-only access.
Outputs	Alert queue and details; updated alert status.
Priority	Medium
Traceability	UC-20 · Alert, Transfer, ComplianceCase · vw_AlertQueue, usp_DismissAlert

FR-25 — Manage Compliance Cases

Description	The system shall support investigation cases that group alerts and record every action until resolution.
Inputs	Title, priority (Low, Medium, High), assigned analyst, alerts; notes; status changes; resolution.

Processing & Rules	1. A case is created from one or more alerts and receives a number such as CASE-2026-0001; it is assigned to its creator unless reassigned. 2. Status flow: Open → In Review → Escalated → In Review; from In Review or Escalated the case ends as Cleared or Closed. 3. Cleared means no suspicious activity: all linked alerts close as FalsePositive. Closed means suspicious activity was confirmed: alerts close as Suspicious. Both require a resolution of at least 20 characters and are final. 4. Escalated cases appear in the administrator's queue; the administrator records a decision note and returns the case to In Review. 5. Notes are append-only. Status changes, alert links, reassignments, and account freezes add notes automatically. 6. Only the assigned analyst changes the status; an administrator may reassign. The analyst may freeze accounts linked to the case (FR-11).
Outputs	Case list, case timeline, updated alert statuses.
Priority	Medium
Traceability	UC-21 · ComplianceCase, CaseNote, Alert · usp_CreateCase, usp_UpdateCase, usp_AddCaseNote

Area 6 — Audit, Reporting and Integrity

FR-26 — Record and View Audit History

Description	The system shall keep an append-only history of important actions and let authorized staff search it.
Inputs	Filters: date range, actor, action, entity type and ID, outcome.
Processing & Rules	1. Audited actions: logins, failures, lockouts, logouts; user and role changes; application submissions and decisions; screening runs and dispositions; account openings and status changes; deposits; completed and rejected transfers; idempotent replays and conflicts; reversals; tier and rule changes; alert creation and disposition; case actions; reconciliation and seed-data runs; report exports; denied access. 2. Each event stores actor (or System), action, entity type and ID, outcome (Success, Denied, Failed), time, and JSON details with old and new values. Passwords, hashes, and session identifiers are never stored. 3. Events are written only by usp_WriteAudit; app_web cannot update or delete them, and a trigger rejects any change. 4. Administrators see all events; compliance analysts see events for entities linked to their cases. Results can be exported to CSV; the export is itself audited.
Outputs	Filtered audit list; CSV export.
Priority	Medium
Traceability	UC-22 · AuditLog · usp_WriteAudit, vw_AuditSearch

FR-27 — View Management Reports

Description	The system shall provide reports and a dashboard computed from database views.
Inputs	Date range (default: last 30 days).
Processing & Rules	1. Reports: • (a) transfer count and value per day; • (b) transfers by status and type, with the top rejection codes; • (c) alerts by rule and status, and alert rate = alerts ÷ completed internal transfers; • (d) cases by status and analyst, and average days to resolution; • (e) balances by account type, and total customer balances compared with the Bank Funding balance; • (f) applications by status and average decision time; • (g) the latest reconciliation result. 2. Visibility: operations officers see (a), (b), (f); compliance analysts see (c), (d); administrators see all. 3. Each report is shown as a table with a chart and can be exported to CSV. 4. No figure is hard-coded; every report reads a view (vw_*).
Outputs	Dashboard and report pages; CSV files.
Priority	High
Traceability	UC-23 · views vw_DailyTransferVolume, vw_TransferStatusSummary, vw_AlertsByRule, vw_CaseSummary, vw_BalanceSummary, vw_ApplicationSummary

FR-28 — Run Reconciliation and Integrity Checks

Description	The system shall check that balances and ledger entries agree and report every difference.
Inputs	Run command (administrator or script).
Processing & Rules	1. Checks: • (a) each account's balance = Σ credits − Σ debits; • (b) each Completed transfer has exactly one debit and one credit, each equal to the transfer amount, on its source and destination; • (c) Rejected transfers have no entries; • (d) each Reversed transfer has exactly one completed reversal; • (e) system-wide Σ debits = Σ credits; • (f) each account's balanceAfter sequence is consistent with its entries in posting order. 2. The run uses SNAPSHOT isolation, so it sees a consistent state without blocking transfers. 3. Results are stored as a ReconciliationRun with one ReconciliationException per difference (type, account or transfer, expected value, actual value). 4. Nothing is repaired automatically; exceptions stay visible until investigated.

Outputs	Run summary (Pass, or number of exceptions) and exception list; status shown on the dashboard.
Priority	High
Traceability	UC-24 · ReconciliationRun, ReconciliationException, LedgerEntry, Account · usp_RunReconciliation

FR-29 — Query Historical Account State

Description	The system should let an administrator see an account's balance and status at a past time and produce a statement for a period.
Inputs	Account; point in time T; or a period T1–T2.
Processing & Rules	1. T must not be earlier than the account's opening time. 2. Balance at T = balanceAfter of the last entry posted at or before T (0.00 if none). 3. A statement for T1–T2 shows the opening balance, every entry, and the closing balance; closing − opening equals the net of the listed entries. 4. Status at T comes from the system-versioned temporal table on Account (FOR SYSTEM_TIME AS OF T) where supported; otherwise it is derived from status-change audit events (C-16). 5. Every query is audited.
Outputs	Point-in-time balance and status; period statement.
Priority	Low
Traceability	UC-25 · LedgerEntry, Account · usp_GetBalanceAsOf, usp_GetStatement

FR-30 — Generate Synthetic Demonstration Data

Description	The system shall generate a repeatable synthetic dataset that includes known suspicious patterns.
Inputs	Random seed; number of customers (default 200); number of days (default 30).
Processing & Rules	1. The same seed always produces the same data. 2. The generator creates fictional customers with approved applications and documents, tiers, accounts of all three types, deposits, and transfers. Every deposit and transfer is executed through usp_RecordDeposit or usp_TransferFunds, so all ledger, limit, and audit rules apply. 3. At least three instances of each fraud-rule pattern are planted; their transfer IDs are stored in the test table test.PlantedPattern so detection can be measured (NFR-09). 4. At least 50 watchlist entries are created, including at least 5 that collide with applicant names. 5. The generator runs only when the database is marked Development and holds no non-generated customers; otherwise it refuses.
Outputs	Populated database; summary of generated counts; audit event with the parameters.
Priority	Medium

Traceability	UC-26 · all classes · usp_GenerateSeedData

3.2 Use Case Model

3.2.1 Use Case Diagrams

The 26 use cases are grouped into four diagrams by functional area. Customer, Operations Officer, Compliance Analyst, and Administrator are specializations of Registered User; Guest is unauthenticated. Generate Fraud Alert is triggered by the system when it extends Send Internal Transfer.

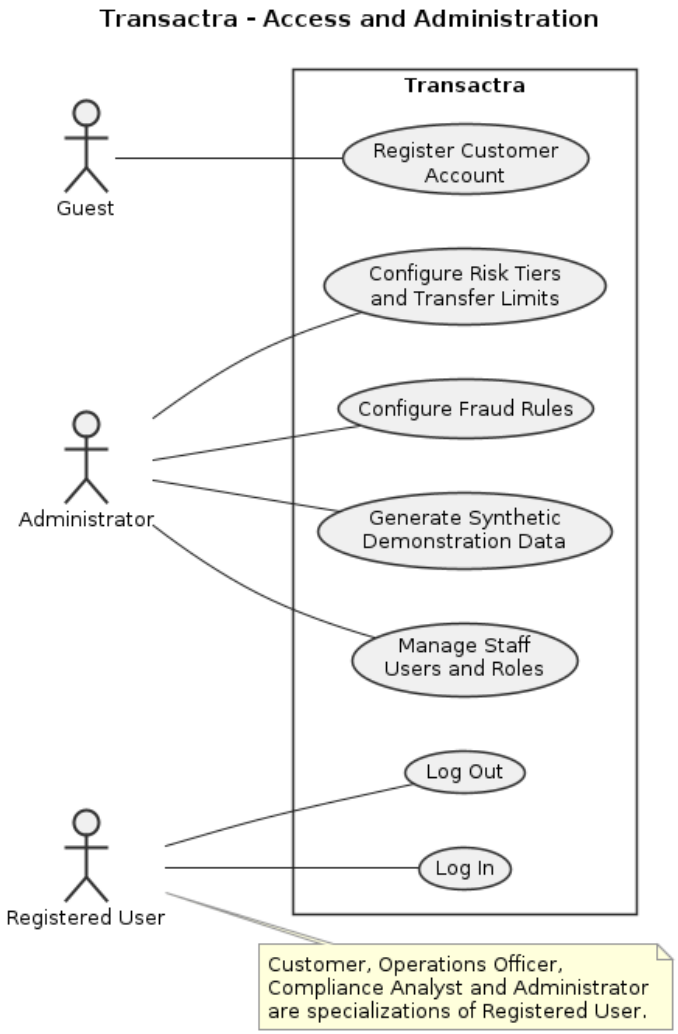


Figure 2: Use-Case Diagram — Access and Administration (UC-01–UC-04, UC-13, UC-18, UC-26)

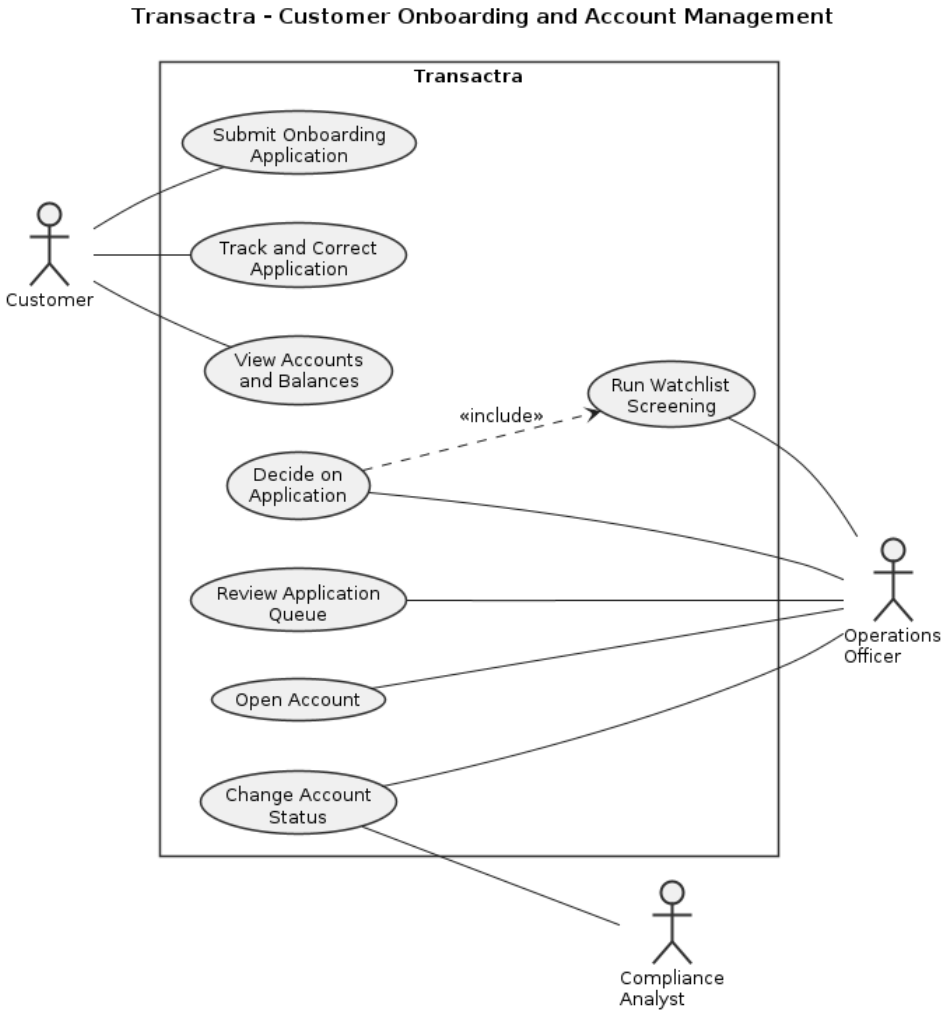


Figure 3: Use-Case Diagram — Customer Onboarding and Account Management (UC-05–UC-12)

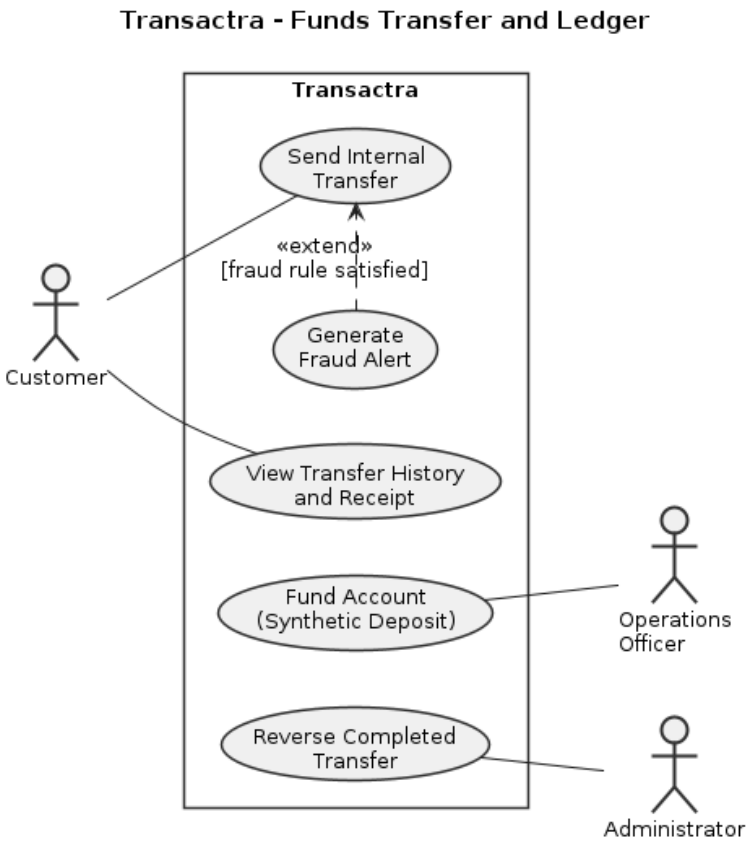


Figure 4: Use-Case Diagram — Funds Transfer and Ledger (UC-14–UC-17, UC-19)

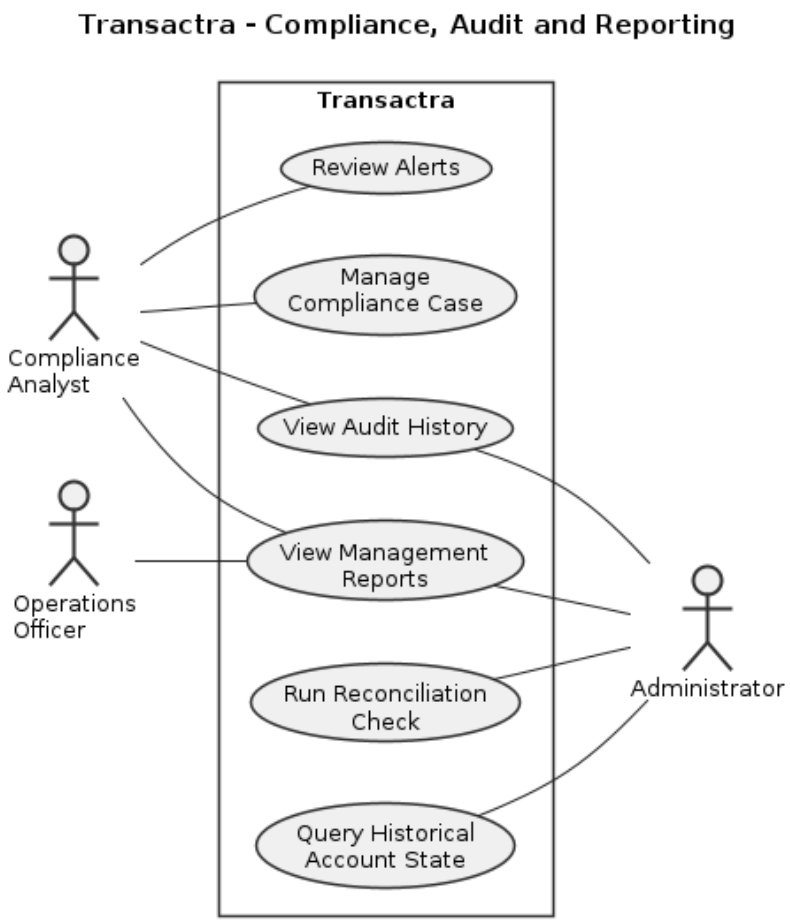


Figure 5: Use-Case Diagram — Compliance, Audit and Reporting (UC-20–UC-25)

3.2.2 Use Case Descriptions

Each use case follows the IEEE template. Rules already specified in the referenced FRs are not repeated; flows refer to them by number.

UC-01: Register Customer Account (FR-01)

Use Case ID	UC-01
Use Case Name	Register Customer Account
Actors	Guest
Description	A visitor creates a customer user so they can apply for an account.
Trigger	The guest selects Register on the home page.
Preconditions	1. The guest is not logged in.
Postconditions	1. A user with the Customer role and status Active exists. 2. The registration is audited.
Normal Flow	1. The system displays the registration form. 2. The guest enters name, email, password, and confirmation, and ticks the simulation notice. 3. The guest submits the form. 4. The system validates the fields and password rules (FR-01). 5. The system checks that the email is not registered. 6. The system creates the user with a hashed password and the Customer role. 7. The system confirms and redirects to the login page.
Alternative Flows	A1. Cancel: the guest leaves the page; nothing is saved.
Exceptions	E1. Invalid or missing field: the field is highlighted with its message; nothing is saved. E2. Email already registered: “This email is already registered”; no user is created. E3. Database failure: a generic error is shown; no user is created.
Priority	High

UC-02: Log In (FR-02, FR-03)

Use Case ID	UC-02
Use Case Name	Log In
Actors	Registered User (Customer, Operations Officer, Compliance Analyst, Administrator)
Description	A registered user authenticates and reaches the home page of their role.
Trigger	The user selects Log In.
Preconditions	1. The user exists and is Active. 2. The user is not logged in.

Postconditions	1. An authenticated session exists. 2. The login is audited.
Normal Flow	1. The user enters email and password. 2. The system checks that the user is not locked. 3. The system verifies the password hash. 4. The system resets the failure counter and creates a session. 5. The system opens the role home page (FR-02 rule 4).
Alternative Flows	A1. Temporary password: a staff user with a temporary password must set a new one before continuing.
Exceptions	E1. Wrong credentials: generic message; failure counter +1. E2. Fifth consecutive failure: the user is locked for 15 minutes; generic message. E3. Locked or inactive user: generic message; no session.
Priority	High

UC-03: Log Out (FR-02)

Use Case ID	UC-03
Use Case Name	Log Out
Actors	Registered User
Description	A user ends their session.
Trigger	The user selects Log Out, or the session times out.
Preconditions	1. The user is logged in.
Postconditions	1. The session is invalid on the server. 2. The logout is audited.
Normal Flow	1. The user selects Log Out. 2. The system invalidates the session and clears the cookie. 3. The system shows the login page with “You have been logged out”.
Alternative Flows	A1. Timeout: after 15 minutes of inactivity the next request is redirected to login with “Your session expired”.
Exceptions	E1. Session already invalid: the system shows the login page.
Priority	High

UC-04: Manage Staff Users and Roles (FR-04)

Use Case ID	UC-04
Use Case Name	Manage Staff Users and Roles
Actors	Administrator

Description	The administrator creates staff users, changes their roles, unlocks users, and activates or deactivates users.
Trigger	The administrator opens Users.
Preconditions	1. The administrator is logged in.
Postconditions	1. User and role changes are saved and audited. 2. Role changes take effect on the user's next request.
Normal Flow	1. The system lists users with roles, status, and last login. 2. The administrator selects Create Staff User. 3. The administrator enters name and email and selects one or more staff roles. 4. The system validates the data and that no Customer role is combined with a staff role. 5. The system creates the user with a temporary password and shows it once. 6. The system confirms and refreshes the list.
Alternative Flows	A1. Change roles: the administrator edits a user's roles; the system applies FR-04 rule 3 and saves. A2. Deactivate: the administrator confirms; sessions end and the user's open cases become Unassigned. A3. Unlock: the system clears lockoutEndAt and the failure counter.
Exceptions	E1. Email exists: rejected. E2. Removing the last Administrator or the administrator's own role: rejected with an explanation. E3. Record changed by another administrator: refused (CR-8).
Priority	Medium

UC-05: Submit Onboarding Application (FR-05)

Use Case ID	UC-05
Use Case Name	Submit Onboarding Application
Actors	Customer
Description	A customer submits synthetic personal details and identity-document metadata for review.
Trigger	The customer selects Apply for an Account.
Preconditions	1. The customer is logged in. 2. The customer has no application in progress and no approved application.
Postconditions	1. A Customer profile (Pending) and an application (Submitted, revision 1) with its documents exist. 2. The application appears in the operations queue (UC-07).

Normal Flow	1. The system shows the application form with a reminder to use fictional data. 2. The customer enters personal details. 3. The customer adds one or more documents (type, reference number, issue and expiry dates). 4. The customer submits. 5. The system validates age, phone, documents, and dates (FR-05). 6. The system saves the profile, application, and documents in one transaction. 7. The system shows the reference and status Submitted.
Alternative Flows	A1. Add proof of address in addition to the identity document.
Exceptions	E1. Under 18, invalid phone, or invalid dates: field messages; nothing saved. E2. No identity document: “Add a National ID card or Passport”. E3. Application already in progress or approved: the system shows the existing application instead.
Priority	High

UC-06: Track and Correct Application (FR-06)

Use Case ID	UC-06
Use Case Name	Track and Correct Application
Actors	Customer
Description	A customer follows the status of their application and corrects it when the officer returns it.
Trigger	The customer opens Application Status.
Preconditions	1. The customer has submitted an application.
Postconditions	1. Status is displayed; or a corrected application is Submitted with revisionNo + 1.
Normal Flow	1. The system shows the application’s status, timeline, and next step. 2. If the status is Returned, the system shows the officer’s reason and an Edit option. 3. The customer corrects the fields or documents and resubmits. 4. The system re-applies FR-05 validation. 5. The system saves the revision and sets the status to Submitted. 6. The system confirms the resubmission.
Alternative Flows	A1. Status Approved: the system links to My Accounts. A2. Status Rejected: the system shows the reason and the date from which a new application is allowed (30 days).
Exceptions	E1. Application not Returned: Edit is not available. E2. Validation errors: field messages; the previous revision stays unchanged.
Priority	High

UC-07: Review Application Queue (FR-07)

Use Case ID	UC-07
Use Case Name	Review Application Queue
Actors	Operations Officer
Description	An officer selects an application from the queue, takes ownership of it, and inspects its details.
Trigger	The officer opens Application Queue.
Preconditions	1. The officer is logged in.
Postconditions	1. The selected application is Under Review and assigned to the officer.
Normal Flow	1. The system lists Submitted and Under Review applications, oldest first. 2. The officer filters the queue if needed and opens an application. 3. The system shows applicant data, document flags, earlier revisions, and screening state. 4. The officer selects Start Review. 5. The system sets Under Review and assigns the officer. 6. The officer continues with screening (UC-08) or a decision (UC-09).
Alternative Flows	A1. Reassign to me: the officer takes over an application assigned to another officer; the change is audited. A2. Customer search: the officer searches customers or accounts in read-only mode.
Exceptions	E1. Another officer started the review first: refused (CR-8); the queue refreshes. E2. Queue empty: “No applications awaiting review”.
Priority	High

UC-08: Run Watchlist Screening (FR-08)

Use Case ID	UC-08
Use Case Name	Run Watchlist Screening
Actors	Operations Officer
Description	The officer screens the applicant against the synthetic watchlist and dispositions any possible matches. It is also included by Decide on Application.
Trigger	The officer selects Run Screening, or UC-09 includes it.
Preconditions	1. The application is Under Review and assigned to the officer.
Postconditions	1. Screening results for the current revision are stored. 2. Each possible match has a disposition, or is pending.

Normal Flow	1. The system normalizes the applicant's name. 2. The system compares it with active watchlist entries (FR-08 rule 2). 3. The system stores the results. 4. The system shows Clear, or the list of possible matches with reasons. 5. For each match the officer selects FalsePositive or TrueMatch and enters a reason. 6. The system saves the dispositions.
Alternative Flows	A1. Clear result: no disposition is needed; the officer proceeds to UC-09.
Exceptions	E1. Missing reason: the disposition is not saved. E2. Screening failure: error shown; no partial results are stored.
Priority	Medium

UC-09: Decide on Application (FR-09)

Use Case ID	UC-09
Use Case Name	Decide on Application
Actors	Operations Officer
Description	The officer approves, rejects, or returns an application. It includes Run Watchlist Screening.
Trigger	The officer selects Decide on an application under review.
Preconditions	1. The application is Under Review and assigned to the officer.
Postconditions	1. The decision, reason, decider, and time are stored. 2. The application and customer statuses are updated; on approval the risk level and tier are set.
Normal Flow	1. If the current revision has no screening, the system runs UC-08. 2. The system shows the decision form. 3. The officer selects Approve, chooses the risk level and tier, and may add a note. 4. The system checks FR-09 rules 3–4. 5. The system saves the decision and sets the application to Approved and the customer to Active. 6. The system offers Open Account (UC-10).
Alternative Flows	A1. Return: the officer enters a reason; the application becomes Returned and the customer can correct it (UC-06). A2. Reject: the officer enters a reason; the application and customer become Rejected.
Exceptions	E1. Undispositioned match, TrueMatch, or expired identity document: Approve is refused with the reason. E2. High risk with Tier 2 or 3: refused. E3. Missing reason for Reject or Return: refused. E4. Application changed meanwhile: refused (CR-8).

Priority	High

UC-10: Open Account (FR-10)

Use Case ID	UC-10
Use Case Name	Open Account
Actors	Operations Officer
Description	The officer opens a checking, savings, or wallet account for an approved customer.
Trigger	The officer selects Open Account on the customer page.
Preconditions	1. The customer is Active.
Postconditions	1. An Active account with balance 0.00 and its subtype row exist. 2. The account appears in the customer's account list.
Normal Flow	1. The system shows the account types still available to the customer. 2. The officer selects a type and its settings. 3. The system checks the per-type limit and the overdraft rule. 4. The system generates the account number and creates the account and subtype in one transaction. 5. The system shows the new account number.
Alternative Flows	A1. The officer continues directly with an initial deposit (UC-14).
Exceptions	E1. Customer not Active: refused. E2. Type limit reached: refused with the current accounts listed. E3. Overdraft for a Tier 1 customer or above 5,000: refused.
Priority	High

UC-11: Change Account Status (FR-11)

Use Case ID	UC-11
Use Case Name	Change Account Status
Actors	Operations Officer, Compliance Analyst (freeze only), Administrator
Description	Staff freeze, unfreeze, or close an account with a reason.
Trigger	The user selects Change Status on an account.
Preconditions	1. The account is not Closed. 2. A compliance analyst may act only on an account linked to a case assigned to them.
Postconditions	1. The new status is stored and audited with the reason.

Normal Flow	1. The system shows the allowed transitions for the user's role. 2. The user selects the new status and enters a reason. 3. The system locks the account row and checks the transition rules. 4. The system updates the status and writes the audit event. 5. The system confirms; for a freeze by an analyst, a note is added to the case.
Alternative Flows	A1. Unfreeze: the R4 monitoring window starts (FR-22).
Exceptions	E1. Close with a non-zero balance or open alerts/cases: refused. E2. Transition not allowed for the role, or the Bank Funding account: refused. E3. Missing reason: refused.
Priority	High

UC-12: View Accounts and Balances (FR-12)

Use Case ID	UC-12
Use Case Name	View Accounts and Balances
Actors	Customer
Description	The customer reviews their accounts, balances, and limits.
Trigger	The customer opens My Accounts.
Preconditions	1. The customer is logged in.
Postconditions	1. Data is displayed; nothing is modified.
Normal Flow	1. The system retrieves the customer's accounts. 2. The system shows the balance and available balance of each account, type-specific limits, remaining daily allowance, and the last five transfers. 3. The customer may open the full history (UC-17) or start a transfer (UC-15).
Alternative Flows	A1. No account yet: the system shows the application status (UC-06).
Exceptions	E1. Retrieval failure: an error is shown instead of incomplete balances.
Priority	High

UC-13: Configure Risk Tiers and Transfer Limits (FR-13)

Use Case ID	UC-13
Use Case Name	Configure Risk Tiers and Transfer Limits
Actors	Administrator
Description	The administrator edits tier limits or moves a customer to another tier.
Trigger	The administrator opens Limits.
Preconditions	1. The administrator is logged in.
Postconditions	1. New limits or the new tier apply to transfers submitted afterwards; the change is audited.

Normal Flow	1. The system shows the three tiers and their limits. 2. The administrator edits values and saves. 3. The system validates FR-13 rule 2. 4. The system saves and confirms.
Alternative Flows	A1. Change a customer's tier: the administrator selects a customer, a new tier, and a reason; the system applies FR-13 rule 4.
Exceptions	E1. Invalid or inconsistent limits: refused with the violated rule. E2. Tier 2 or 3 for a High-risk customer: refused.
Priority	Medium

UC-14: Fund Account (Synthetic Deposit) (FR-14)

Use Case ID	UC-14
Use Case Name	Fund Account (Synthetic Deposit)
Actors	Operations Officer
Description	The officer records a synthetic incoming payment into a customer account.
Trigger	The officer selects Record Deposit.
Preconditions	1. The destination account is Active.
Postconditions	1. A Deposit transfer with one debit (Bank Funding) and one credit (customer) is committed; balances are updated.
Normal Flow	1. The system shows the deposit form with a new idempotency key. 2. The officer enters the account, amount, and description and confirms. 3. The system executes usp_RecordDeposit (FR-17–FR-19). 4. The system shows the reference and new balance.
Alternative Flows	A1. Repeated submission: the original result is returned (FR-19).
Exceptions	E1. Invalid amount, inactive account, or wallet maximum exceeded: rejected with the code; no money moves.
Priority	High

UC-15: Send Internal Transfer (FR-15 – FR-19)

Use Case ID	UC-15
Use Case Name	Send Internal Transfer
Actors	Customer
Description	The customer sends money to another Transactra account. The system validates, executes atomically, posts double entries, and protects against duplicates. It is extended by Generate Fraud Alert.
Trigger	The customer selects Send Money.

Preconditions	1. The customer has at least one Active account.
Postconditions	1. Success: one Completed transfer, one debit and one credit, updated balances, stored idempotency result, audit event. 2. Failure: no ledger change; a Rejected transfer with its code for business-rule failures.
Normal Flow	1. The system shows the transfer form with a new idempotency key. 2. The customer selects the source account and enters the destination number, amount, and description. 3. The system looks up the destination and shows the confirmation page (FR-15). 4. The customer confirms. 5. The system calls usp_TransferFunds, which claims the key, locks both accounts in ID order, and runs all checks (FR-16). 6. The system inserts the transfer and the two ledger entries and updates both balances (FR-17, FR-18). 7. The system commits and shows the receipt. 8. After the commit, the system evaluates the fraud rules (extension point: UC-19).
Alternative Flows	A1. Edit before confirming: the customer returns to the form; the same key is kept. A2. Duplicate request (refresh, double click, network retry): the system returns the original result without executing again.
Exceptions	E1. A check fails: Rejected with its code (e.g., INSUFFICIENT_FUNDS); no money moves. E2. Same key with different details: IDEMPOTENCY_CONFLICT. E3. Deadlock: retried up to 3 times with the same key; if it still fails, “Transfer failed — no money was moved”. E4. Destination not found: shown at step 3; the transfer cannot be confirmed.
Priority	High

UC-16: Reverse Completed Transfer (FR-20)

Use Case ID	UC-16
Use Case Name	Reverse Completed Transfer
Actors	Administrator
Description	The administrator cancels the financial effect of a completed transfer with a compensating transfer.
Trigger	The administrator selects Reverse on a transfer.
Preconditions	1. The transfer is Completed, of type Internal or Deposit, not reversed, and at most 30 days old.
Postconditions	1. A Reversal transfer with its two entries exists; the original is Reversed; balances are updated.

Normal Flow	1. The system shows the original transfer and the effect of the reversal. 2. The administrator enters a reason and confirms. 3. The system executes usp_ReverseTransfer with FR-17–FR-19. 4. The system shows the reversal reference.
Alternative Flows	A1. Repeated submission: the original reversal result is returned.
Exceptions	E1. Not eligible (already reversed, too old, or a reversal): refused. E2. Original destination has insufficient funds or is Closed: REVERSAL_INSUFFICIENT_FUNDS; no change.
Priority	Medium

UC-17: View Transfer History and Receipt (FR-21)

Use Case ID	UC-17
Use Case Name	View Transfer History and Receipt
Actors	Customer
Description	The customer searches their transfer history and opens receipts.
Trigger	The customer opens Transfer History.
Preconditions	1. The customer is logged in.
Postconditions	1. Data is displayed; nothing is modified.
Normal Flow	1. The system lists the latest transfers involving the customer's accounts. 2. The customer applies filters. 3. The system shows matching transfers, 25 per page. 4. The customer opens a transfer. 5. The system shows the receipt (FR-21 rule 3), which can be printed.
Alternative Flows	A1. Search by reference: the system opens that transfer directly if it belongs to the customer.
Exceptions	E1. No results: "No transfers match your filters". E2. Invalid filter (e.g., date range over 12 months): message; the search is not run. E3. Reference of another customer: "Not found".
Priority	High

UC-18: Configure Fraud Rules (FR-22)

Use Case ID	UC-18
Use Case Name	Configure Fraud Rules
Actors	Administrator
Description	The administrator activates, deactivates, or tunes the fraud rules.

Trigger	The administrator opens Fraud Rules.
Preconditions	1. The administrator is logged in.
Postconditions	1. Rule changes apply to later evaluations and are audited.
Normal Flow	1. The system lists the four rules with description, parameters, severity, state, and alerts raised in the last 30 days. 2. The administrator edits a rule and saves. 3. The system validates the ranges (FR-22 rule 2). 4. The system saves and confirms.
Alternative Flows	A1. Deactivate or activate a rule with one action.
Exceptions	E1. Value out of range: refused. E2. Rule changed meanwhile: refused (CR-8).
Priority	Medium

UC-19: Generate Fraud Alert (FR-23)

Use Case ID	UC-19
Use Case Name	Generate Fraud Alert
Actors	System (triggered by UC-15 or a batch run); Compliance Analyst receives the result
Description	The system evaluates the active rules after a transfer and creates or updates an alert when a rule is satisfied. It extends Send Internal Transfer under the condition [fraud rule satisfied].
Trigger	A completed internal transfer commits, or an administrator starts a batch evaluation.
Preconditions	1. At least one rule is active.
Postconditions	1. A New alert linked to the rule, account, and transfers exists, or the transfer is linked to an existing open alert.
Normal Flow	1. The system runs usp_EvaluateFraudRules for the transfer. 2. For each active rule, it counts or sums the relevant transfers in the window. 3. A threshold is reached. 4. The system checks for an open alert of the same rule and account with an overlapping window. 5. None exists, so the system creates an alert with severity and summary and links the transfers. 6. The alert appears in the alert queue (UC-20).
Alternative Flows	A1. Matching open alert exists: the transfer is linked to it. A2. High-risk customer: severity is raised one level.
Exceptions	E1. Evaluation error: logged; the next batch run re-evaluates; the transfer remains committed.
Priority	Medium

UC-20: Review Alerts (FR-24)

Use Case ID	UC-20
Use Case Name	Review Alerts
Actors	Compliance Analyst
Description	The analyst examines alerts and dismisses them or adds them to a case.
Trigger	The analyst opens Alert Queue.
Preconditions	1. The analyst is logged in.
Postconditions	1. The alert is Closed (FalsePositive) or Under Review in a case.
Normal Flow	1. The system lists New alerts by severity, then age. 2. The analyst opens an alert. 3. The system shows the rule, linked transfers, customer risk level and tier, and recent alerts on the account. 4. The analyst selects Add to Case and chooses a new or existing open case. 5. The system links the alert and sets it Under Review (continues in UC-21).
Alternative Flows	A1. Dismiss: the analyst enters a reason; the alert becomes Closed with disposition FalsePositive.
Exceptions	E1. Missing reason: refused. E2. Alert already handled by another analyst: refused (CR-8).
Priority	Medium

UC-21: Manage Compliance Case (FR-25)

Use Case ID	UC-21
Use Case Name	Manage Compliance Case
Actors	Compliance Analyst; Administrator (escalations, reassignment)
Description	The analyst investigates a case, records notes and actions, and resolves it.
Trigger	The analyst creates a case from alerts or opens an assigned case.
Preconditions	1. The case is not Cleared or Closed. 2. The analyst is assigned to the case.
Postconditions	1. All actions are recorded as notes and audit events. 2. A resolved case is final and its alerts are Closed with a disposition.

Normal Flow	1. The system shows the case summary, linked alerts and transfers, and the timeline. 2. The analyst sets the status to In Review and adds investigation notes. 3. The analyst freezes an involved account if needed (UC-11). 4. The analyst selects Close (suspicious activity confirmed) and enters a resolution. 5. The system closes the case and marks its alerts Suspicious.
Alternative Flows	A1. Clear: no suspicious activity; alerts close as FalsePositive. A2. Escalate: the case goes to the administrator, who adds a decision note and returns it to In Review. A3. Link more alerts: the analyst adds open alerts from the queue. A4. Reassign: an administrator assigns another analyst.
Exceptions	E1. Resolution shorter than 20 characters: refused. E2. User is not the assigned analyst: refused. E3. Case changed meanwhile: refused (CR-8).
Priority	Medium

UC-22: View Audit History (FR-26)

Use Case ID	UC-22
Use Case Name	View Audit History
Actors	Administrator; Compliance Analyst (case-related events only)
Description	Staff search the audit history to see who did what, when, and with what result.
Trigger	The user opens Audit History.
Preconditions	1. The user is an Administrator or Compliance Analyst.
Postconditions	1. Events are displayed; an export is itself audited.
Normal Flow	1. The user sets filters (date range, actor, action, entity, outcome). 2. The system returns matching events within the user's visibility, newest first. 3. The user opens an event to see its old and new values.
Alternative Flows	A1. Export: the system produces a CSV file of the filtered events.
Exceptions	E1. No results: message. E2. Analyst requests events outside their cases: those events are not returned.
Priority	Medium

UC-23: View Management Reports (FR-27)

Use Case ID	UC-23
Use Case Name	View Management Reports
Actors	Administrator; Operations Officer and Compliance Analyst (their report subsets)

Description	Staff view the dashboard and reports for a date range.
Trigger	The user opens Dashboard or Reports.
Preconditions	1. The user holds a staff role.
Postconditions	1. Reports are displayed from views; nothing is modified.
Normal Flow	1. The system selects the last 30 days. 2. The system queries the views permitted for the user's role. 3. The system shows each report as a table and a chart. 4. The user changes the date range; the system refreshes the reports.
Alternative Flows	A1. Export a report to CSV (audited).
Exceptions	E1. No data in range: "No activity in this period". E2. Query failure: error instead of partial figures.
Priority	High

UC-24: Run Reconciliation Check (FR-28)

Use Case ID	UC-24
Use Case Name	Run Reconciliation Check
Actors	Administrator
Description	The administrator verifies that balances and ledger entries agree.
Trigger	The administrator selects Run Reconciliation.
Preconditions	1. No other reconciliation run is in progress.
Postconditions	1. A run with its exceptions (if any) is stored; nothing is repaired.
Normal Flow	1. The system starts usp_RunReconciliation under SNAPSHOT isolation. 2. The system performs checks (a)–(f) of FR-28. 3. The system stores the run and any exceptions. 4. The system shows Pass, or the exception list with expected and actual values.
Alternative Flows	A1. View an earlier run from the run history.
Exceptions	E1. A run is already in progress: the user is told to wait. E2. Check failure: the run is stored as Failed with the error code.
Priority	High

UC-25: Query Historical Account State (FR-29)

Use Case ID	UC-25
Use Case Name	Query Historical Account State
Actors	Administrator

Description	The administrator sees an account's balance and status at a past time or a statement for a period.
Trigger	The administrator opens Account History.
Preconditions	1. The account exists.
Postconditions	1. Results are displayed; the query is audited.
Normal Flow	1. The administrator selects an account and a time T. 2. The system validates that T is not before the opening time. 3. The system computes the balance at T and retrieves the status at T. 4. The system shows both values.
Alternative Flows	A1. Statement: the administrator enters T1–T2; the system shows the opening balance, entries, and closing balance.
Exceptions	E1. T before opening or in the future: refused. E2. Temporal table unavailable: the status is derived from audit events, and this is stated on the page.
Priority	Low

UC-26: Generate Synthetic Demonstration Data (FR-30)

Use Case ID	UC-26
Use Case Name	Generate Synthetic Demonstration Data
Actors	Administrator
Description	The administrator fills a development database with repeatable synthetic data and planted patterns.
Trigger	The administrator selects Generate Data (or runs the seed script).
Preconditions	1. The database is marked Development and holds no non-generated customers.
Postconditions	1. Customers, accounts, deposits, transfers, watchlist entries, and planted patterns exist; batch fraud evaluation has run.
Normal Flow	1. The administrator enters the seed, number of customers, and number of days. 2. The system checks the preconditions. 3. The system generates customers, applications, and accounts. 4. The system executes deposits and transfers through the standard procedures. 5. The system plants patterns and records them in test.PlantedPattern. 6. The system runs batch fraud evaluation (UC-19) and shows the generated counts.
Alternative Flows	A1. Reset: in Development only, the administrator first runs the documented reset script.

Exceptions	E1. Preconditions not met: refused; nothing is generated. E2. Failure part-way: the current batch is rolled back and the error is reported.
Priority	Medium

3.3 Conceptual Class Model

3.3.1 Conceptual Class Diagrams

The model has 22 classes. It is drawn in two parts for readability; Account and User appear in both, with their attributes shown in part (a).

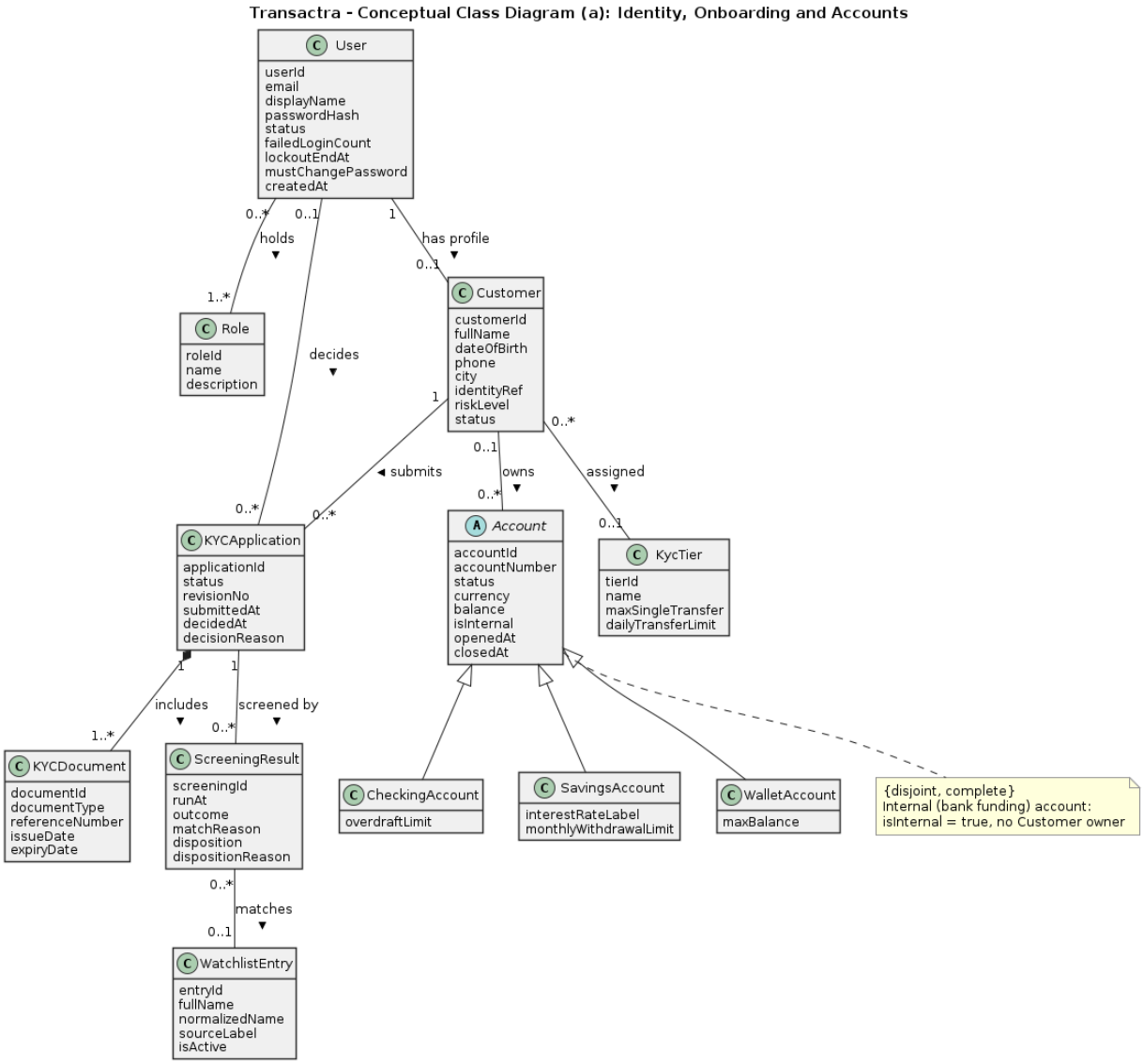


Figure 6: Conceptual Class Diagram (a) — Identity, Onboarding and Accounts

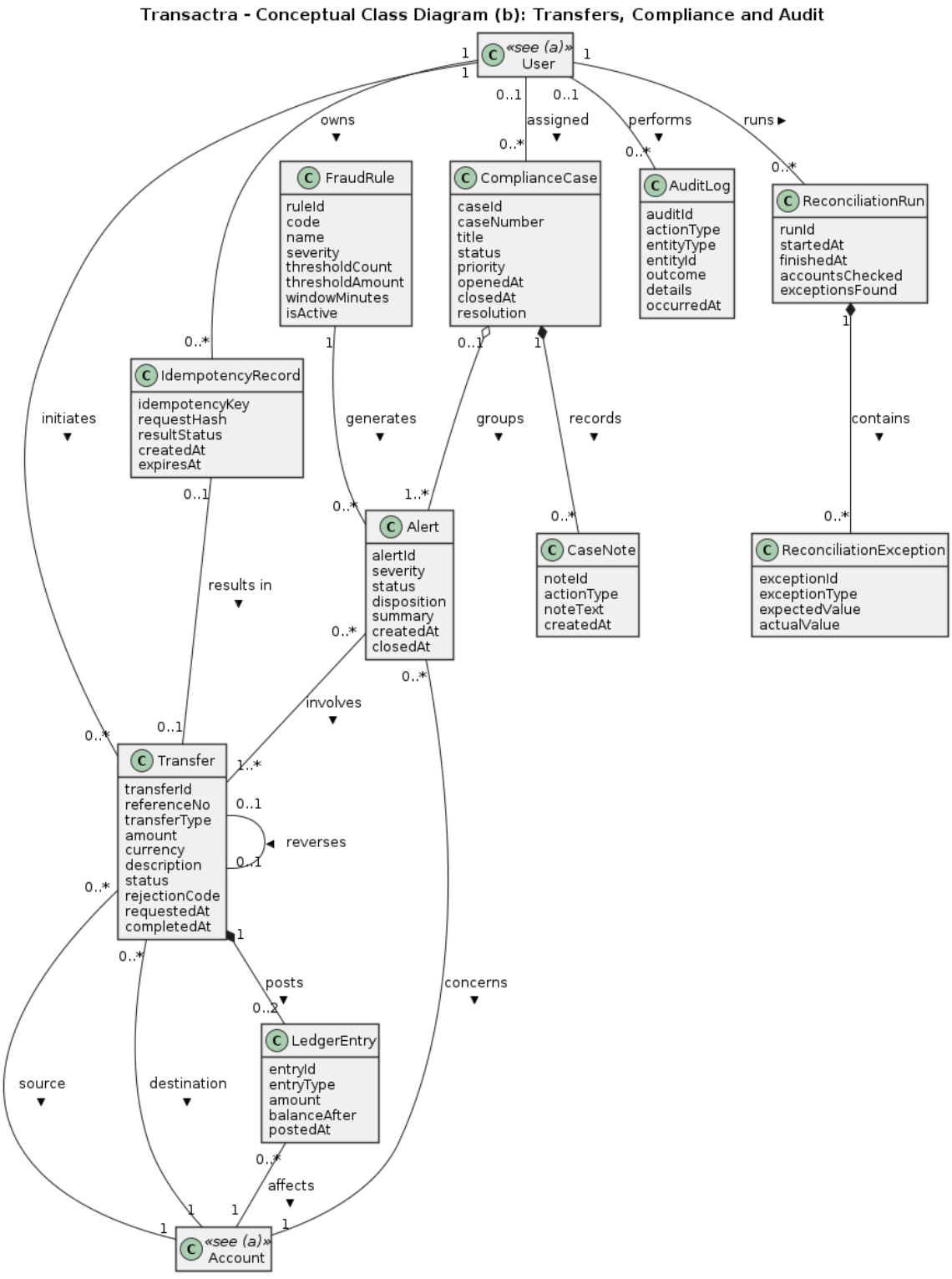


Figure 7: Conceptual Class Diagram (b) — Transfers, Compliance and Audit

3.3.2 Class Descriptions

The table gives each class's purpose and associations with multiplicities. Attributes, data types, and constraints are specified once, in Appendix A.

Class	Purpose	Associations (multiplicity)
User	Login identity of a customer or staff member.	Holds 1..* Role; has 0..1 Customer profile; initiates 0..* Transfer; owns 0..* IdempotencyRecord; decides 0..* KYCApplication; is assigned 0..* ComplianceCase; performs 0..* AuditLog; runs 0..* ReconciliationRun.
Role	Permission group (Customer, OperationsOfficer, ComplianceAnalyst, Administrator).	Held by 0..* User.
Customer	Profile of a person who applies for and owns accounts.	Belongs to 1 User; submits 0..* KYCApplication; owns 0..* Account; is assigned 0..1 KycTier.
KycTier	Limit policy of a KYC tier.	Applies to 0..* Customer.
KYCApplication	Onboarding request, its revisions, and its decision.	Submitted by 1 Customer; includes 1..* KYCDocument (composition); screened by 0..* ScreeningResult; decided by 0..1 User.
KYCDocument	Metadata of one synthetic identity or address document.	Part of exactly 1 KYCApplication.
WatchlistEntry	Fictional name on the screening list.	Matched by 0..* ScreeningResult.
ScreeningResult	Outcome of one screening comparison and the officer's disposition.	Belongs to 1 KYCApplication; matches 0..1 WatchlistEntry.
Account (abstract)	Common data of every account.	Owned by 0..1 Customer (none for Bank Funding); source and destination of 0..* Transfer; affected by 0..* LedgerEntry; concerned by 0..* Alert. Specialized into exactly one subtype {disjoint, complete}.
CheckingAccount	Account with an optional overdraft.	Is an Account.
SavingsAccount	Account with a monthly withdrawal limit.	Is an Account.
WalletAccount	Account with a maximum balance.	Is an Account.
Transfer	Business-level money movement and its outcome (Internal, Deposit, Reversal).	1 source and 1 destination Account; posts 0 or 2 LedgerEntry (composition); initiated by 1 User; reverses 0..1 Transfer; involved in 0..* Alert; result of 0..1 IdempotencyRecord.
LedgerEntry	Immutable debit or credit.	Belongs to 1 Transfer; affects 1 Account.

Class	Purpose	Associations (multiplicity)
IdempotencyRecord	Remembers a request key, its fingerprint, and its result.	Owned by 1 User; results in 0..1 Transfer.
FraudRule	Explainable detection rule with parameters.	Generates 0..* Alert.
Alert	One firing of a fraud rule.	Generated by 1 FraudRule; concerns 1 Account; involves 1..* Transfer; grouped in 0..1 ComplianceCase.
ComplianceCase	Investigation of one or more alerts.	Groups 1..* Alert (aggregation); records 0..* CaseNote (composition); assigned to 0..1 User.
CaseNote	Append-only investigation entry or automatic action record.	Belongs to 1 ComplianceCase; written by 0..1 User (none when system-generated).
AuditLog	Append-only record of an important action.	Performed by 0..1 User (none when the actor is System).
ReconciliationRun	One execution of the integrity checks.	Run by 1 User; contains 0..* ReconciliationException (composition).
ReconciliationException	One difference found by a run.	Belongs to 1 ReconciliationRun; concerns 0..1 Account and 0..1 Transfer.

3.3.3 Mapping Notes for the Relational Schema

- **Specialization:** Account is mapped with class-table inheritance. Each subtype table shares the primary key accountId; a discriminator accountType plus a composite foreign key (accountId, accountType) guarantees that each account has exactly one matching subtype row.
- **Many-to-many associations:** User–Role becomes UserRole; Alert–Transfer becomes AlertTransfer.
- **Self-association:** Transfer.reversalOfTransferId is a nullable foreign key with a filtered unique index, so a transfer can be reversed at most once.
- **Reserved word:** User is a T-SQL reserved word, so the table is named AppUser. With ASP.NET Core Identity (integer keys), AppUser and UserRole are implemented by extending AspNetUsers and AspNetUserRoles.
- **Immutability:** LedgerEntry, AuditLog, and CaseNote accept INSERT only; on SQL Server 2022 they may also be created as append-only ledger tables (C-16).

3.4 Non-Functional Requirements

Each NFR states the requirement, a measurable target, how it is verified, and its acceptance criteria. Test scripts and results are stored in /tests and summarized in the final report.

ID	Quality Attribute	Measurable Target (summary)
NFR-01	Transactional integrity	0 partial transfers in all fault-injection tests.
NFR-02	Ledger consistency	0 reconciliation exceptions on clean data; every planted corruption detected; report totals match.
NFR-03	Concurrency safety	No overspending or inconsistent ledger under 50 concurrent transfers on one account.
NFR-04	Reliability and idempotency	Exactly one transfer per idempotency key under replays and double submission.
NFR-05	Security	No plaintext credentials, no successful SQL injection or CSRF, no direct DML by app_web.
NFR-06	Authorization	100% of prohibited role and record-access attempts denied.
NFR-07	Privacy	Only fictional data; counterparty data masked in every customer view.
NFR-08	Auditability	Every audited action produces one complete event; audit rows cannot be changed.
NFR-09	Performance and benchmarking	P95 ≤ 1 s for core pages on the benchmark dataset; documented index experiment; 100% planted-pattern detection.
NFR-10	Usability and accessibility	≥ 90% unaided task completion; Lighthouse accessibility ≥ 85; works on all browser/viewport combinations.
NFR-11	Maintainability and portability	Clean build from README in ≤ 60 minutes; scripts re-runnable.

NFR-01 — Transactional Integrity

Requirement	All financial writes shall run through the approved stored procedures inside one transaction, so that a transfer is either fully committed (transfer, two entries, balances, idempotency result, audit event) or leaves no trace except a Rejected record.
Target	0 partial transfers in 100% of fault-injection tests.
Verification	1. A T-SQL test harness calls usp_TransferFunds with a test-only parameter @FailAfterStep (enabled only in the test database) for every step of FR-17. 2. Further cases: a constraint violation on the second ledger entry, and a session killed in the middle of a transaction. 3. After each case, row counts of Transfer, LedgerEntry, and IdempotencyRecord and both balances are compared with the values before the call.

Acceptance Criteria	• For every failure point, counts and balances equal the values before the call. • The procedure returns a failure code and the UI shows “no money was moved”. • No ledger entry exists without its transfer.

NFR-02 — Ledger Consistency and Calculation Accuracy

Requirement	Every committed transfer shall balance, maintained balances shall equal ledger-derived balances, and every report figure shall equal an independent query.
Target	0 reconciliation exceptions on clean data; 100% of planted corruptions detected; 100% of report totals match.
Verification	1. Run usp_RunReconciliation after seed generation and after every test suite. 2. In an isolated test database, plant known corruptions as db_owner (change a balance, delete one entry with the trigger disabled, add an entry to a Rejected transfer) and run reconciliation. 3. Compare each FR-27 report with a hand-written validation query.
Acceptance Criteria	• Clean data gives 0 exceptions. • Each planted corruption is reported with the correct exception type, account or transfer, and values. • All report totals equal their validation queries.

NFR-03 — Concurrency Safety

Requirement	Concurrent requests on the same accounts shall never overspend, leave an inconsistent ledger, or end in an unresolved deadlock.
Target	• 50 concurrent transfers of EGP 100 from a Tier 2 checking account holding EGP 2,000 (no overdraft): exactly 20 Completed and 30 INSUFFICIENT_FUNDS, final balance 0.00. • 100 crossing transfers (A→B and B→A in parallel): all complete or are rejected by rules; no deadlock remains unresolved after retries.
Verification	1. A C# test client opens independent connections and starts the requests together (Parallel.ForEach); each scenario runs 10 times. 2. The runs are repeated with READ COMMITTED + UPDLOCK and with SERIALIZABLE, recording throughput, deadlocks, and retries. 3. Reconciliation runs after each run.
Acceptance Criteria	• All 10 runs of each scenario meet the target. • Reconciliation reports 0 exceptions. • The chosen isolation strategy is justified with the measurements in the benchmark report.

NFR-04 — Reliability and Idempotency

Requirement	A failed operation shall return a clear result and leave a consistent state; a repeated request shall never create a second financial operation.

Target	• 100 replays of one key (sequential and 20 concurrent) → 1 transfer, 2 entries. • 100% of conflicting key reuses rejected. • Double click and page refresh in the UI → 1 transfer.
Verification	1. Scripted replays with the same key and the same details, then with different details. 2. UI test: double-click Confirm, refresh the result page, and use Back and resubmit. 3. Count transfers and entries per key.
Acceptance Criteria	• Exactly one transfer per key. • Every replay returns the original reference and status. • Conflicts return IDEMPOTENCY_CONFLICT and change nothing.

NFR-05 — Security

Requirement	Transactra shall protect credentials, sessions, and data: hashed passwords (C-08), parameterized queries (C-09), a least-privilege database login (C-04), HTTPS, secure cookies, anti-forgery tokens on all POST forms, login lockout (FR-02), and error messages without internal details (C-13).
Target	0 plaintext credentials; 0 successful SQL injections or CSRF requests; 0 successful direct DML statements by app_web on Transfer, LedgerEntry, or AuditLog.
Verification	1. Inspect the user table for password storage. 2. Enter injection strings (e.g., ' OR 1=1 --) in every text input and search field. 3. Connect as app_web in SSMS and attempt INSERT, UPDATE, and DELETE on the protected tables. 4. Send a POST without its anti-forgery token. 5. Make 5 failed logins and check the lockout. 6. Optionally run an OWASP ZAP baseline scan.
Acceptance Criteria	• All checks behave as expected: permission errors for app_web, HTTP 400 without a token, lockout after 5 failures. • No High-risk finding in the ZAP baseline (if run).

NFR-06 — Authorization

Requirement	Each role shall be able to perform only the actions in the permission matrix (2.3.1), and customers shall reach only their own records. Enforcement is on the server and in the procedures, never only by hiding buttons.
Target	100% of prohibited attempts in the authorization test matrix denied, with no data disclosed.
Verification	1. A test matrix calls every protected endpoint as each of the five user classes. 2. Insecure-direct-object-reference tests change account, transfer, application, and case IDs in URLs and form fields. 3. Procedures are called directly with a wrong @ActorUserId.

Acceptance Criteria	• Every prohibited request returns Access denied or Not found. • Target records are unchanged. • Every denial is audited.

NFR-07 — Privacy

Requirement	Transactra shall use fictional data only, collect the minimum data needed, store no document images, and mask counterparty data in customer views.
Target	0 real personal data; counterparty name and number masked in 100% of customer screens.
Verification	1. Review the seed generator and forms: no upload controls and no binary columns. 2. Review the transfer confirmation, history, and receipt screens. 3. Check that the README and forms state that data is synthetic.
Acceptance Criteria	• The privacy checklist passes completely.

NFR-08 — Auditability

Requirement	Every action listed in FR-26 shall be traceable to its actor, time, operation, entity, and outcome, and audit records shall be append-only.
Target	100% of listed actions produce exactly one complete event; 0 successful modifications of audit rows.
Verification	1. A test script performs each listed action and queries AuditLog for the matching event. 2. UPDATE and DELETE on AuditLog are attempted as app_web and as db_owner.
Acceptance Criteria	• Each action has one event with all required fields and no secrets. • All modification attempts fail.

NFR-09 — Performance and Benchmarking

Requirement	Core pages shall respond quickly on a documented dataset, and performance claims shall be backed by reproducible measurements.
Target	• Benchmark dataset of 100,000 transfers, 10 concurrent users: 95th-percentile server response time ≤ 1 s for login, account overview, transfer submission, history (first page), and alert queue; ≤ 3 s for the dashboard. • Reconciliation of the benchmark dataset ≤ 30 s. • Index experiment for at least two report queries, reporting logical reads and elapsed time before and after indexing. • Fraud rules detect 100% of planted patterns; the number of extra alerts is reported.
Verification	1. Apache JMeter (or k6) with 10 virtual users. 2. SET STATISTICS IO, TIME ON and actual execution plans for the queries. 3. Compare alerts with test.PlantedPattern.

Acceptance Criteria	• All targets are met. • The benchmark report documents hardware, dataset size, scripts, plans, and results, and does not present synthetic detection results as real-world fraud performance.

NFR-10 — Usability and Accessibility

Requirement	The UI shall be organized by task, use clear labels and field-level validation messages, ask for confirmation before financial and destructive actions, work with the keyboard, and adapt to mobile and desktop screens.
Target	• ≥ 90% of tasks completed without help by at least 5 test users. • Send Money takes ≤ 60 s on average. • Lighthouse accessibility score ≥ 85 on the main pages. • All core tasks work in Chrome, Firefox, and Edge at 360 × 640 and 1366 × 768.
Verification	1. A usability session with customer tasks (register, apply, view balance, send money, find a receipt) and staff tasks (approve an application, review an alert, open a case). 2. Lighthouse audits and a keyboard-only walkthrough. 3. A browser and viewport checklist.
Acceptance Criteria	• All targets are met. • Any blocking issue is fixed and retested before the final demonstration.

NFR-11 — Maintainability and Portability

Requirement	Schema, procedures, views, seed data, and tests shall be separate, numbered, re-runnable scripts (C-15). Database objects shall follow naming conventions (usp_, vw_, trg_, PK_, FK_, CK_, UQ_, IX_). The README shall state the versions and the steps to build and run the system.
Target	A team member who did not set up the project builds and runs it from the README on a clean machine in ≤ 60 minutes; running the deployment scripts twice raises no errors.
Verification	1. Clean-machine (or fresh VM) build test. 2. Run the deployment scripts twice. 3. Run the full test suite.
Acceptance Criteria	• The build succeeds within the time limit. • The second run raises no errors. • All tests pass.

3.5 External Interface Requirements

3.5.1 User Interfaces

The UI is a responsive, English-language Bootstrap 5 web interface (C-11, C-12). Every page has a role-specific navigation bar, success and error messages after each action, field-level validation messages, and confirmation dialogs before financial or irreversible actions. Monetary values are right-aligned with two decimals and “EGP”.

Customer Portal

- **Register / Log In:** forms with validation; generic login error.
- **Application:** multi-section form (personal details, documents) and a status page with a timeline and next step.
- **My Accounts:** cards per account with number, type, status, balance, available balance, and limits.
- **Send Money:** form → confirmation page with the masked recipient name → result page with reference or reason.
- **Transfer History:** filterable table with direction, signed amount, and balance after; printable receipt.

Operations Console

- **Application Queue** with filters, application detail, screening panel with match dispositions, decision form.
- **Customer search** (read-only), Open Account form, Record Deposit form, Change Status dialog with a required reason.

Compliance Console

- **Alert Queue** sorted by severity with filters, alert detail with linked transfers.
- **Cases:** list, case page with timeline, notes, linked alerts, status actions, and Freeze Account.

Administrator Area

- **Dashboard** with report tiles and charts (Chart.js) and the latest reconciliation result.
- **Users and Roles, Limits, Fraud Rules, Reversal, Reconciliation, Account History, Audit History, Generate Data.**

3.5.2 Hardware Interfaces

Transactra has no hardware interfaces. Users work with the standard input and display devices of a laptop, desktop, tablet, or phone. Server hardware is the team’s development machine or the classroom environment.

3.5.3 Software Interfaces

- **Microsoft SQL Server 2019/2022:** stores all persistent data. Features used: transactions with XACT_ABORT, UPDLOCK/ROWLOCK hints, SNAPSHOT isolation

(ALLOW_SNAPSHOT_ISOLATION ON) for reconciliation, filtered unique indexes, CHECK/FOREIGN KEY/UNIQUE constraints, triggers, JSON functions; optional system-versioned temporal tables and ledger tables (C-16).

- **ASP.NET Core 8 MVC and ASP.NET Core Identity:** request handling, views, authentication, password hashing, lockout, roles, anti-forgery tokens.
- **Microsoft.Data.SqlClient / Dapper:** executes stored procedures with typed parameters and maps result sets.
- **Bootstrap 5 and Chart.js:** layout and dashboard charts (static files served by the application).
- **External systems:** none. Transactra exposes no public API and calls no external service (C-07).

The application uses the database through the following programmatic interface. Only these objects are granted to app_web.

Database Object	Purpose	Used by
usp_SubmitApplication / usp_ResubmitApplication	Create or revise an application with its documents.	FR-05, FR-06
usp_RunScreening / usp_DispositionMatch	Screen an application; record match dispositions.	FR-08
usp_DecideApplication	Approve, reject, or return; set risk level and tier.	FR-09
usp_OpenAccount / usp_ChangeAccountStatus	Create an account with its subtype; change status.	FR-10, FR-11
usp_UpdateTier / usp_ChangeCustomerTier	Maintain tier limits; re-tier a customer.	FR-13
usp_LookupAccount	Return existence and masked name of a destination.	FR-15
usp_TransferFunds / usp_RecordDeposit / usp_ReverseTransfer	Validate and execute transfers atomically with double entry and idempotency.	FR-14, FR-16–FR-20
usp_UpdateFraudRule / usp_EvaluateFraudRules	Maintain rules; evaluate after a transfer or in batch.	FR-22, FR-23
usp_DismissAlert / usp_CreateCase / usp_UpdateCase / usp_AddCaseNote	Alert disposition and case workflow.	FR-24, FR-25
usp_ManageUser	Create staff users, change roles, activate, deactivate, unlock.	FR-04
usp_WriteAudit	Append an audit event (called inside other procedures).	FR-26
usp_RunReconciliation	Run integrity checks and store results.	FR-28
usp_GetBalanceAsOf / usp_GetStatement	Point-in-time balance and period statement.	FR-29
usp_GenerateSeedData	Generate the synthetic dataset (Development only).	FR-30

Database Object	Purpose	Used by
vw_* views	Read models for queues, history, audit search, and reports.	FR-07, FR-12, FR-21, FR-24, FR-26, FR-27

3.5.4 Communication Interfaces

- **Browser ↔ application:** HTTPS (TLS 1.2 or later) with HSTS; HTML pages, and JSON only for chart data; cookies are HttpOnly, Secure, and SameSite=Strict.
- **Application ↔ database:** SQL Server TDS protocol on TCP 1433 (or shared memory locally), with Encrypt=True; TrustServerCertificate is allowed only in development. The connection string is kept in user secrets or environment variables, never in the repository.
- **Other protocols:** none. No e-mail, SMS, FTP, message queue, or third-party network calls.

4. Appendices

Appendix A: Data Dictionary

This appendix defines every attribute of the conceptual model (Section 3.3) with its SQL Server type and constraints. Foreign keys, junction tables, discriminators, and rowversion columns are listed even though the conceptual diagrams omit them, so the relational schema can be derived directly. Abbreviations: PK primary key, FK foreign key, UQ unique, CK check, DF default. All timestamps default to SYSDATETIME().

A.1 AppUser (class User)

Column	SQL Type	Constraints / Description
userId	INT IDENTITY	PK.
email	NVARCHAR(256)	NOT NULL; UQ; stored in lowercase.
displayName	NVARCHAR(100)	NOT NULL; 2–100 characters.
passwordHash	NVARCHAR(512)	NOT NULL; Identity PBKDF2 hash (C-08).
status	VARCHAR(8)	CK IN (Active, Inactive); DF Active.
failedLoginCount	TINYINT	DF 0; CK 0–5.
lockoutEndAt	DATETIME2(0)	NULL when not locked.
mustChangePassword	BIT	DF 0; 1 for new staff users.
createdAt	DATETIME2(0)	NOT NULL.

A.2 Role and A.3 UserRole

Column	SQL Type	Constraints / Description
Role.roleId	TINYINT	PK.
Role.name	VARCHAR(20)	UQ; CK IN (Customer, OperationsOfficer, ComplianceAnalyst, Administrator).
Role.description	NVARCHAR(200)	NULL.
UserRole.userId	INT	PK part; FK → AppUser.
UserRole.roleId	TINYINT	PK part; FK → Role. The Customer role cannot coexist with a staff role (checked by usp_ManageUser).
UserRole.assignedAt / assignedById	DATETIME2(0) / INT	NOT NULL / FK → AppUser, NULL for self-registration.

A.4 Customer

Column	SQL Type	Constraints / Description
customerId	INT IDENTITY	PK.
userId	INT	NOT NULL; UQ; FK → AppUser.
fullName	NVARCHAR(100)	NOT NULL.

Column	SQL Type	Constraints / Description
dateOfBirth	DATE	NOT NULL; age ≥ 18 checked by usp_SubmitApplication.
phone	CHAR(11)	CK LIKE '01[0125]' followed by 8 digits.
city	NVARCHAR(50)	NOT NULL.
identityRef	CHAR(14)	UQ; 14 digits; synthetic only.
riskLevel	VARCHAR(6)	NULL until approval; CK IN (Low, Medium, High).
tierId	TINYINT	NULL until approval; FK → KycTier; CK NOT (riskLevel = High AND tierId > 1).
status	VARCHAR(8)	CK IN (Pending, Active, Rejected).
rowVer	ROWVERSION	Optimistic concurrency (CR-8).

A.5 KycTier

Column	SQL Type	Constraints / Description
tierId	TINYINT	PK; CK IN (1, 2, 3).
name	VARCHAR(10)	UQ (Tier 1, Tier 2, Tier 3).
maxSingleTransfer	DECIMAL(18,2)	CK > 0.
dailyTransferLimit	DECIMAL(18,2)	CK maxSingleTransfer ≤ dailyTransferLimit ≤ 1,000,000.
rowVer	ROWVERSION	Optimistic concurrency.

A.6 KYCApplication

Column	SQL Type	Constraints / Description
applicationId	INT IDENTITY	PK.
referenceNo	AS 'APP-' + 6-digit id	Computed, persisted; UQ.
customerId	INT	NOT NULL; FK → Customer. Filtered UQ index on customerId WHERE status IN (Submitted, UnderReview, Returned) enforces one application in progress.
status	VARCHAR(11)	CK IN (Submitted, UnderReview, Returned, Approved, Rejected).
revisionNo	SMALLINT	DF 1; CK ≥ 1.
submittedAt	DATETIME2(0)	NOT NULL.
assignedOfficerId	INT	NULL; FK → AppUser.
decidedById / decidedAt	INT / DATETIME2(0)	NULL until a decision; FK → AppUser; CK both NOT NULL when status IN (Approved, Rejected, Returned).
decisionReason	NVARCHAR(500)	CK NOT NULL when status IN (Rejected, Returned).
rowVer	ROWVERSION	Optimistic concurrency.

A.7 KYCDocument

Column	SQL Type	Constraints / Description
documentId	INT IDENTITY	PK.
applicationId	INT	NOT NULL; FK → KYCApplication ON DELETE NO ACTION.
documentType	VARCHAR(14)	CK IN (NationalID, Passport, ProofOfAddress); UQ (applicationId, documentType).
referenceNumber	NVARCHAR(30)	NOT NULL; synthetic.
issueDate / expiryDate	DATE / DATE	CK issueDate < expiryDate.

A.8 WatchlistEntry

Column	SQL Type	Constraints / Description
entryId	INT IDENTITY	PK.
fullName	NVARCHAR(100)	NOT NULL; fictional.
normalizedName	NVARCHAR(100)	NOT NULL; IX for matching.
sourceLabel	NVARCHAR(50)	NOT NULL (e.g., Synthetic List A).
isActive	BIT	DF 1.

A.9 ScreeningResult

Column	SQL Type	Constraints / Description
screeningId	INT IDENTITY	PK.
applicationId / revisionNo	INT / SMALLINT	FK → KYCApplication; revision screened.
runAt / runById	DATETIME2(0) / INT	NOT NULL; FK → AppUser.
outcome	VARCHAR(13)	CK IN (Clear, PossibleMatch).
entryId	INT	FK → WatchlistEntry; CK NULL when Clear, NOT NULL when PossibleMatch.
matchReason	NVARCHAR(100)	NULL when Clear.
disposition	VARCHAR(13)	NULL until decided; CK IN (FalsePositive, TrueMatch).
dispositionReason / dispositionById	NVARCHAR(500) / INT	CK NOT NULL when disposition is set; FK → AppUser.

A.10 Account

Column	SQL Type	Constraints / Description
accountId	INT IDENTITY	PK; (accountId, accountType) UQ for subtype FKs.
accountNumber	CHAR(12)	UQ; CK 12 digits starting 10, 20, or 30.
customerId	INT	FK → Customer; CK NULL if and only if isInternal = 1.
accountType	VARCHAR(8)	Discriminator; CK IN (Checking, Savings, Wallet).
status	VARCHAR(6)	CK IN (Active, Frozen, Closed); DF Active.

Column	SQL Type	Constraints / Description
currency	CHAR(3)	DF EGP; CK = EGP.
balance	DECIMAL(18,2)	DF 0.00; maintained snapshot (A-09); CK isInternal = 1 OR balance ≥ −5,000.00 as a safety net.
isInternal	BIT	DF 0; 1 only for Bank Funding.
openedAt / closedAt	DATETIME2(0)	closedAt CK NOT NULL if and only if status = Closed.
rowVer	ROWVERSION	Optimistic concurrency for status changes.
validFrom / validTo	DATETIME2 GENERATED ALWAYS	Optional system versioning (FR-29, C-16).

A.11 CheckingAccount, SavingsAccount, WalletAccount

Column	SQL Type	Constraints / Description
accountId (each subtype)	INT	PK; FK (accountId, accountType) → Account with accountType fixed by CK to the subtype.
CheckingAccount.overdraftLimit	DECIMAL(18,2)	DF 0; CK 0–5,000.
SavingsAccount.interestRateLabel	NVARCHAR(20)	Display only (A-10).
SavingsAccount.monthlyWithdrawalLimit	TINYINT	DF 6; CK 1–31.
WalletAccount.maxBalance	DECIMAL(18,2)	DF 50,000; CK > 0 and ≤ 1,000,000.

A.12 Transfer

Column	SQL Type	Constraints / Description
transferId	BIGINT IDENTITY	PK.
referenceNo	VARCHAR(20)	UQ; TRX-YYYYMMDD-NNNNNN.
transferType	VARCHAR(8)	CK IN (Internal, Deposit, Reversal).
sourceAccountId / destinationAccountId	INT / INT	FK → Account; CK source ≠ destination; IX (sourceAccountId, requestedAt), IX (destinationAccountId, requestedAt).
amount	DECIMAL(18,2)	CK > 0 and ≤ 1,000,000.
currency	CHAR(3)	DF EGP.
description	NVARCHAR(140)	NULL.
status	VARCHAR(9)	CK IN (Completed, Rejected, Reversed).
rejectionCode	VARCHAR(30)	CK NOT NULL if and only if status = Rejected.
initiatedById	INT	NOT NULL; FK → AppUser.
reversalOfTransferId	BIGINT	FK → Transfer; filtered UQ WHERE NOT NULL; CK NOT NULL if and only if transferType = Reversal.
requestedAt / completedAt	DATETIME2(3)	completedAt NULL for Rejected.

A.13 LedgerEntry

Column	SQL Type	Constraints / Description
entryId	BIGINT IDENTITY	PK; also gives posting order.
transferId	BIGINT	NOT NULL; FK → Transfer; UQ (transferId, entryType).
accountId	INT	NOT NULL; FK → Account; IX (accountId, entryId) INCLUDE (entryType, amount, balanceAfter, postedAt).
entryType	VARCHAR(6)	CK IN (Debit, Credit).
amount	DECIMAL(18,2)	CK > 0.
balanceAfter	DECIMAL(18,2)	NOT NULL.
postedAt	DATETIME2(3)	NOT NULL. Table is insert-only (trigger trg_LedgerEntry_Immutable).

A.14 IdempotencyRecord

Column	SQL Type	Constraints / Description
userId / idempotencyKey	INT / UNIQUEIDENTIFIER	Composite PK; FK → AppUser.
operationType	VARCHAR(8)	CK IN (Internal, Deposit, Reversal).
requestHash	BINARY(32)	SHA-256 fingerprint (FR-19).
resultStatus / resultCode	VARCHAR(10) / VARCHAR(30)	CK resultStatus IN (InProgress, Completed, Rejected).
transferId	BIGINT	NULL while InProgress; FK → Transfer.
createdAt / expiresAt	DATETIME2(3)	expiresAt = createdAt + 24 hours; IX on expiresAt for purge.

A.15 FraudRule

Column	SQL Type	Constraints / Description
ruleId	TINYINT	PK.
code	VARCHAR(20)	UQ (VELOCITY, FAN_IN, HIGH_VALUE_WINDOW, POST_UNFREEZE).
name / description	NVARCHAR(60) / NVARCHAR(300)	NOT NULL.
severity	VARCHAR(6)	CK IN (Low, Medium, High).
thresholdCount	INT	NULL or CK 1–100.
thresholdAmount	DECIMAL(18,2)	NULL or CK 1–1,000,000.
windowMinutes	INT	CK 1–10,080.
isActive / rowVer	BIT / ROWVERSION	DF 1.

A.16 Alert and A.17 AlertTransfer

Column	SQL Type	Constraints / Description
alertId	INT IDENTITY	PK.
ruleId / accountId	TINYINT / INT	FK → FraudRule / FK → Account.
caseId	INT	NULL; FK → ComplianceCase.
severity	VARCHAR(6)	CK IN (Low, Medium, High).
status	VARCHAR(11)	CK IN (New, UnderReview, Closed); IX (status, severity, createdAt).
disposition	VARCHAR(13)	CK IN (FalsePositive, Suspicious); NOT NULL if and only if status = Closed.
summary	NVARCHAR(300)	Values and thresholds at firing time.
windowStart / windowEnd	DATETIME2(0)	Evaluation window (deduplication, FR-23).
createdAt / closedAt / closedById	DATETIME2(0) / DATETIME2(0) / INT	closedById FK → AppUser.
AlertTransfer.alertId / transferId	INT / BIGINT	Composite PK; FKs → Alert, Transfer.

A.18 ComplianceCase and A.19 CaseNote

Column	SQL Type	Constraints / Description
caseId	INT IDENTITY	PK.
caseNumber	VARCHAR(15)	UQ; CASE-YYYY-NNNN.
title	NVARCHAR(100)	NOT NULL.
status	VARCHAR(10)	CK IN (Open, InReview, Escalated, Cleared, Closed).
priority	VARCHAR(6)	CK IN (Low, Medium, High).
assignedToId / openedById	INT / INT	FK → AppUser; assignedToId NULL when Unassigned.
openedAt / closedAt	DATETIME2(0)	closedAt CK NOT NULL if and only if status IN (Cleared, Closed).
resolution	NVARCHAR(1000)	CK LEN ≥ 20 when status IN (Cleared, Closed).
rowVer	ROWVERSION	Optimistic concurrency.
CaseNote.noteId / caseId / authorId	INT / INT / INT	PK / FK → ComplianceCase / FK → AppUser (NULL = System).
CaseNote.actionType	VARCHAR(14)	CK IN (Note, StatusChange, AlertLinked, Reassigned, AccountFrozen, Escalation).
CaseNote.noteText / createdAt	NVARCHAR(1000) / DATETIME2(0)	Insert-only.

A.20 AuditLog

Column	SQL Type	Constraints / Description
auditId	BIGINT IDENTITY	PK.

Column	SQL Type	Constraints / Description
actorUserId	INT	NULL = System; FK → AppUser.
actionType	VARCHAR(40)	NOT NULL (e.g., TransferCompleted, LoginFailed).
entityType / entityId	VARCHAR(30) / VARCHAR(30)	IX (entityType, entityId).
outcome	VARCHAR(7)	CK IN (Success, Denied, Failed).
details	NVARCHAR(MAX)	NULL or CK ISJSON(details) = 1; never contains secrets.
occurredAt	DATETIME2(3)	IX. Table is insert-only (trigger trg_AuditLog_Immutable).

A.21 ReconciliationRun and A.22 ReconciliationException

Column	SQL Type	Constraints / Description
runId	INT IDENTITY	PK.
runById	INT	FK → AppUser.
startedAt / finishedAt	DATETIME2(0)	finishedAt NULL while running.
status	VARCHAR(10)	CK IN (Running, Pass, Exceptions, Failed); filtered UQ WHERE status = Running (one run at a time).
accountsChecked / transfersChecked / exceptionsFound	INT	CK ≥ 0.
Exception.exceptionId / runId	INT / INT	PK / FK → ReconciliationRun.
Exception.exceptionType	VARCHAR(20)	CK IN (BalanceMismatch, UnbalancedTransfer, EntriesOnRejected, MissingReversal, GlobalImbalance, BalanceChainBreak).
Exception.accountId / transferId	INT / BIGINT	NULL or FK → Account / Transfer.
Exception.expectedValue / actualValue	DECIMAL(18,2)	NULL when not numeric.

A.23 Derived Values (calculated, not stored)

Value	Definition
Ledger balance	Σ credits − Σ debits of an account; must equal Account.balance (FR-28).
Available balance	balance + overdraftLimit for an Active checking account; balance for other Active accounts; 0.00 if Frozen or Closed.
Daily used amount	Σ amount of the source account's Completed Internal transfers today (FR-16 check 8).
Savings withdrawals this month	Count of Completed outgoing Internal transfers from a savings account in the current calendar month.
Balance at T	balanceAfter of the last entry with postedAt ≤ T; 0.00 if none (FR-29).
Alert rate	Number of alerts ÷ number of Completed Internal transfers in the period (FR-27).

Appendix B: Glossary

Terms already defined in Section 1.3 are not repeated.

Term	Definition
95th percentile (P95)	The response time within which 95% of measured requests complete.
Anti-forgery token	A per-form secret that prevents another site from submitting forms on a user's behalf (CSRF).
Bank Funding account	The internal account that is the source of all synthetic deposits; its negative balance equals total net deposits.
Class-table inheritance	Mapping a superclass and each subclass to separate tables that share the same primary key.
Deadlock	Two transactions waiting for each other's locks; SQL Server ends one of them with error 1205.
Discriminator	A column (accountType) that states which subtype a row belongs to.
Fan-in	Many different accounts sending money to the same account within a short period (rule R2).
Filtered index	An index over only the rows that meet a WHERE condition; used to enforce conditional uniqueness.
Fingerprint	A SHA-256 hash of the request details stored with an idempotency key.
Least privilege	Granting a login only the permissions it needs (app_web: EXECUTE on procedures, SELECT on views).
Lock ordering	Always locking rows in the same order (ascending accountId) to prevent deadlocks.
Masked name	A name showing only initials (e.g., O*** A***), used to confirm a recipient without disclosing their identity.
Optimistic concurrency	Detecting conflicting edits with a rowversion value instead of holding locks while a user edits.
Planted pattern	A suspicious sequence deliberately inserted into synthetic data, with its transfers recorded for measuring detection.
SNAPSHOT isolation	An isolation level in which a transaction reads a consistent version of the data as of its start without blocking writers.
Temporal table	A SQL Server table that automatically keeps the history of row changes and can be queried AS OF a past time.
Ledger table	A SQL Server 2022 table whose history is cryptographically protected; optional in Transactra.
Tipping-off	Revealing to a customer that they are being screened or investigated; avoided by hiding screening results and alerts from customers.
UPDLOCK	A table hint that takes update locks when reading rows, so that other writers wait until the transaction ends.

Term	Definition
Velocity	The number of transactions in a short time window (rule R1).
XACT_ABORT	A T-SQL setting that rolls back the whole transaction when a run-time error occurs.